ISPRS Open Journal of Photogrammetry and Remote Sensing 19 (2026) 100117

Contents lists available at ScienceDirect

ISPRS Open Journal of Photogrammetry and Remote Sensing

journal homepage: www.journals.elsevier.com/isprs-open-journal-of-photogrammetry-and-remote-sensing

Benchmarking for practice: Few-shot time-series crop-type classification on the EuroCropsML dataset

Joana Reuss [a,*], Jan Macdonald [d], Simon Becker [d,e], Ekaterina Gikalo [a], Konrad Schultka [d], Lorenz Richter [d,f], Marco Körner [a,b,c]

[a] Technical University of Munich (TUM), TUM School of Engineering and Design, Department of Aerospace and Geodesy, Chair of Remote Sensing Technology, Munich, 80333, Germany
[b] Technical University of Munich (TUM), Munich Data Science Institute (MDSI), Garching, 85748, Germany
[c] ELLIS Unit Jena, University of Jena, Jena, 07743, Germany
[d] dida Datenschmiede GmbH, Berlin, 10827, Germany
[e] ETH Zurich, Department of Mathematics, Zurich, 8004, Switzerland
[f] Zuse Institute Berlin, Berlin, 14195, Germany

ARTICLE INFO

Keywords:
Transfer and meta-learning
Few-shot learning
Self-supervised learning
Crop-type classification
EuroCropsML

ABSTRACT

Accurate crop-type classification from satellite time series is essential for agricultural monitoring. Consequently, various *machine learning (ML)* algorithms, aiming on enhancing classification performance on data-scarce tasks, have been developed. While previous evaluations demonstrated the effectiveness of these algorithms in certain situations, these studies frequently lacked real-world scenarios. Hence, the performance of the algorithms in challenging practical applications has not yet been profoundly evaluated. To facilitate future research in this domain, we present the first comprehensive benchmark for evaluating supervised and *self-supervised learning (SSL)* methods for crop-type classification under real-world conditions. This benchmark study relies on the EuroCropsML time-series dataset, which combines farmer-reported crop data with Sentinel-2 satellite observations from Estonia, Latvia, and Portugal. Our findings indicate that variants of *Model-Agnostic Meta-Learning (MAML; Finn et al., 2017)* achieve slightly higher accuracy compared to supervised transfer learning and SSL. For instance, algorithms belonging to the MAML-family show on average a 31% higher overall accuracy for a 20-shot benchmark task when compared to regular transfer learning. However, compared to simpler transfer learning, the improvement of meta-learning comes at the cost of increased computational demands and training time. Moreover, supervised methods benefit most when pre-trained and fine-tuned on geographically close regions. In addition, while SSL generally lags behind meta-learning, it demonstrates advantages over training from scratch — particularly in capturing fine-grained features essential for real-world crop-type classification — and also surpasses standard transfer learning. This highlights its practical value when labeled pre-training crop data is scarce. Our insights highlight the trade-offs between accuracy and computational demand in selecting supervised machine learning methods for real-world crop-type classification tasks and underscore the difficulties of knowledge transfer across diverse geographic regions. Furthermore, they demonstrate the practical value of SSL approaches when labeled pre-training crop data is scarce. The corresponding code is publicly available at https://github.com/jsreu/EurocropsML-Meta-Learning and https://github.com/jsreu/EuroCropsML-SSL.

1. Introduction

Satellites continuously generate spatio-temporal remote sensing data of the Earth's surface, which forms a cornerstone of modern *Earth observation (EO)* techniques. They play an important role in a very wide range of applications, from hurricane forecasting (Boussioux et al., 2022) or methane detection (Kumar et al., 2020) to vegetation analysis (Odenweller and Johnson, 1984; Reed et al., 1994). In particular, satellite imagery has been used in agriculture for fertilization analysis (Santaga et al., 2021), harvest detection (Marković et al., 2023), as well as yield forecast and crop assessment (Bekmukhamedov et al., 2024).

* Corresponding author.
E-mail address: joana.reuss@tum.de (J. Reuss).

https://doi.org/10.1016/j.ophoto.2026.100117
Received 18 August 2025; Received in revised form 5 January 2026; Accepted 18 January 2026
Available online 19 February 2026
2667-3932/© 2026 The Authors. Published by Elsevier B.V. on behalf of International Society of Photogrammetry and Remote Sensing. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

A common key challenge in these tasks is the efficient extraction of relevant and valuable information from the abundance of collected remote sensing data. Recently, *artificial intelligence (AI)*, particularly ML and *deep learning (DL)* methods, have shown a significant impact on EO and remote sensing, and can address these challenges across entire data processing chains (Tuia et al., 2024).

Crop data plays a vital role in supporting decision-making processes in various economic sectors related to food supply, agriculture, the environment, and forestry, as well as socioeconomic development (Gómez et al., 2016). Analyzing and understanding the distribution of crop types in different regions is essential for making informed decisions about food security, infrastructure development, agricultural management, and precision farming (Bekmukhamedov et al., 2024; Gómez et al., 2016). Furthermore, variations in crop data are used directly in research on climate change (Song et al., 2021). While ML methods have recently achieved strong results in this domain Qi et al., Saini and Ghosh, the availability of comprehensive crop data is crucial to fully leverage the potential benefits of data-driven ML methods. However, availability varies significantly across different geographical regions, reflecting a widespread Eurocentric and Amerocentric bias in spatio-temporal datasets (Shankar et al., 2017), which hinders the adoption of ML-based data processing in data-scarce regions until now. Accurate few-shot crop classification enables farmers and agricultural organizations to move beyond reactive management. High-performing classification models, validated under real-world conditions, are essential for implementing precision farming and for supporting governmental and institutional bodies in efficient *Common Agriculture Policy (CAP)* subsidy verification and reliable national food security forecasting.

Transfer and *meta-learning* approaches offer promising directions for addressing the challenges posed by such data imbalances. Transfer learning refers to the approach of generalizing insights obtained from one machine learning task (typically data-rich) to another related task (typically data-scarce) in order to improve the performance on the second task. More broadly, meta-learning refers to methods that attempt to improve the generalization capabilities of machine learning models by considering multiple different but related tasks (each of which individually could be data-scarce). A general comparison of the two approaches was recently made by Dumoulin et al. (2021), see also Section 2 for more details.

However, labeled crop data remain costly to obtain Shankar et al. (2017). This makes learning frameworks that can effectively exploit unlabeled satellite imagery particularly valuable. SSL addresses this need by enabling models to learn useful data representations without requiring expensive annotations. In remote sensing, where vast amounts of unlabeled time-series data are readily available, SSL appears to be particularly well-suited to uncover complex spatial, spectral, and temporal patterns—making it a compelling alternative in regions with limited or no labeled data.

Despite the growing interest in these approaches, there is currently a notable gap in the thorough analysis and comparison of transfer, meta-, and self-supervised learning techniques for real-world multi-class crop-type classification using spatio-temporal satellite data. Moreover, the advantages and disadvantages of these techniques have not been thoroughly examined. In this work, we provide such an analysis. More precisely, we evaluate the potential of various existing supervised and self-supervised learning approaches for few-shot time-series crop-type classification by benchmarking their performance on the EuroCropsML dataset (Reuss et al., 2025b). The dataset extends the EuroCrops reference data (Schneider et al., 2023b) with Sentinel-2 L1C reflectance data and is publicly available (Reuss and Macdonald, 2024). We investigate how both supervised and SSL-based pre-training influence classification performance in real-world settings with limited annotated data, and assess the challenges of transferring learned representations across diverse geographical, climatic, and agricultural conditions. Our study establishes a realistic trial case to evaluate existing and future crop-monitoring algorithms, specifically focusing on their performance in few-shot regimes. Moreover, it demonstrates how to utilize the EuroCropsML dataset as a real-world benchmark.

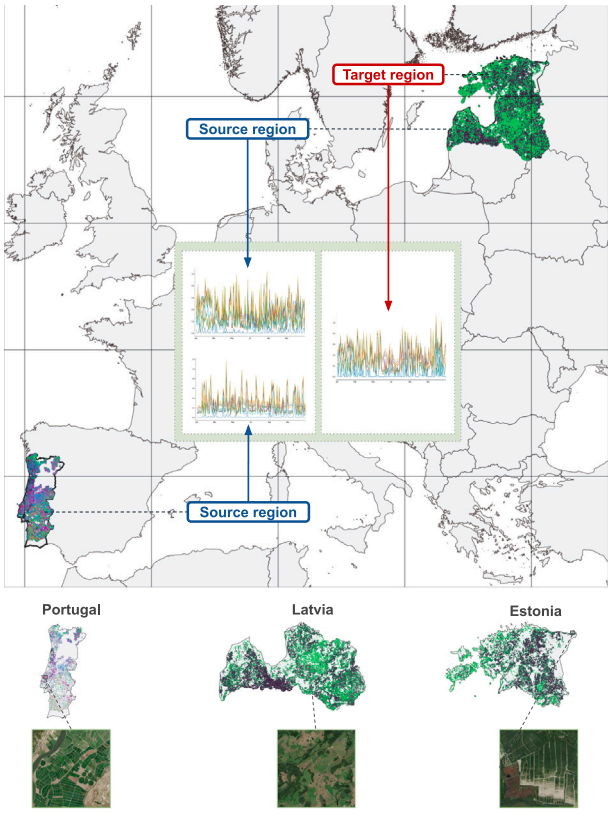


Fig. 1. Visualization of crop fields (using EuroCrops *Hierarchical Crop and Agriculture Taxonomy (HCAT; Schneider et al., 2021, 2023b)* level 3) in the source and target region(s). The initial supervised pre-training is conducted on the Sentinel-2 L1C agricultural time series of the source region(s), followed by a process of fine-tuning and evaluation of the models on a distinct target region.

Using a state-of-the-art *Transformer* encoder architecture (Vaswani et al., 2017), each supervised learning approach is initially trained on a designated source region and subsequently evaluated on a geographically distinct target region, as visualized in Fig. 1. This experimental setup enables the assessment of model generalization under domain shifts. We examine the following approaches, described in more detail in Section 4:

Transfer Learning Transfer learning consists of a two-step procedure of pre-training an ML model that gets subsequently fine-tuned.

MAML The *Model-Agnostic Meta-Learning* approach introduced by Finn et al. (2017) and further investigated by Raghu et al. (2019), Ye and Chao (2021) is a very flexible and widely used optimization-based meta-learning algorithm (*cf.* Section 4.2.1).

FOMAML The *First-Order Model-Agnostic Meta-Learning* algorithm is a computationally more efficient variant of the original MAML framework, proposed by Finn et al. (2017) (*cf.* Section 4.2.2).

ANIL The *Almost No Inner Loop* algorithm introduced by Raghu et al. (2019) is another computationally more efficient variant of MAML (*cf.* Section 4.2.3).

TIML The *Tasked-Informed Meta-Learning* algorithm was introduced by Tseng et al. (2022) and enriches MAML by including additional task-specific information (*e.g.*, geographical coordinates), *cf.* Section 4.2.4.

Furthermore, we compare three self-supervised learning pre-trained *Masked Autoencoders (MAEs; He et al., 2022)*:

PRESTO The ***P**retrained **R**emote **S**ening **T**ransf**o**rmer* model Tseng et al. (2023) is a pixel-wise MAE and serves as our SSL baseline.

PRESTO-XTS Our variant *PRESTO-E**X**tended **T**emporal and **S**pectral* adapts the original PRESTO model by expanding its temporal and spectral resolution, *cf.* Section 4.3.4.

CrossPRESTO-XTS *Cross-Attention PRESTO-XTS* is a computationally more efficient version of PRESTO-XTS, *cf.* Section 4.3.4.

Both of our SSL variants are pre-trained from scratch on combined European Sentinel-1 and Sentinel-2 imagery from the SEN12MS-CR-TS dataset, along with ***E**uropean **Re**Analysis Version **5** (ERA5; Hersbach et al., 2020)* reanalysis temperature and precipitation data. Beyond the original benchmark, for SSL, we also analyze the effect of incorporating additional data during fine-tuning, matching the input modalities during pre-training. In particular, we complement the EUROCROPSML Sentinel-2 Estonia (target region) data with Sentinel-1 radar data and ERA5 reanalysis temperature and precipitation measurements, as outlined in Section 5.2.

To establish an overall baseline, we also train a randomly initialized Transformer-based attention model directly on the target region data, allowing us to quantify the benefits of knowledge transfer through the above pre-training approaches.

By providing a thorough practical benchmark for few-shot time-series crop-type classification, we aim to support future research and facilitate practical applications in agricultural remote sensing. In summary, our main findings comparing the aforementioned algorithms on the EUROCROPSML benchmark dataset are:

1. Meta-learning algorithms of the MAML family tend to slightly outperform other algorithms, such as the baseline (no pre-training), transfer learning (standard pre-training), algorithms of the *TIML* family, and SSL methods in terms of prediction accuracy and Cohen's kappa score.
2. Despite having additional location information available, the algorithms of the *TIML* family show no improved performance.
3. The meta-learning algorithms tend to require longer overall training times and more computational resources compared to the baselines and self-supervised learning.
4. Meta-learning algorithms require more extensive tuning of hyperparameters, which has a detrimental effect on their marginally superior prediction performance.
5. Transferring pre-gained knowledge geographically poses a significant challenge for all investigated supervised algorithms.
6. Pre-training on geographically and spectrally distant data introduces stronger domain shifts, potentially degrading overall accuracy but also regularizing against dominance by specific classes.
7. Self-supervised learning offers clear advantages over training from scratch — particularly for minority classes — underscoring the potential of SSL when labeled pre-training data is unavailable or scarce and highlighting the need for more targeted representation learning approaches in agricultural applications.

2. Related work

Recent studies have explored both supervised and self-supervised methods for crop-type classification, aiming to reduce reliance on labeled data and enhance model performance across diverse settings. This paper contributes to a better understanding of the performance of transfer, meta-, and self-supervised learning algorithms on different data diversities specific to remote sensing and crop classification. It presents a benchmark on a real-world crop-type dataset with varying diversity levels, allowing an in-depth analysis of ML algorithms for Earth observation. We start by reviewing different ML approaches commonly used for this task before discussing available crop datasets.

2.1. Supervised learning

Supervised learning remains the dominant paradigm for crop-type classification, due to the availability of labeled agricultural datasets that enable models to learn discriminative spectral–temporal patterns. *Convolutional Neural Networks (CNNs)* have been widely applied to extract spatial features from satellite and proximal imagery for tasks such as land-cover mapping, crop-type identification, and damage assessment (Kussul et al., 2017; Simón Sánchez et al., 2022; Dhande and Malik, 2022). Beyond convolutional architectures, recent studies show that models with temporal reasoning capabilities better accommodate irregular sampling and noise, as demonstrated by Rußwurm and Körner (2020), who found that self–attention–based and recurrent architectures outperform purely convolutional models on raw satellite time series. Hybrid approaches have also been explored: Chamorro Martinez et al. (2021) combined convolutional encoders with recurrent components and reported competitive performance across several crop-type benchmarks. However, these methods still exhibit sensitivity to class imbalance and regional variability, underscoring the need for more robust learning strategies.

Supervised pre-training methods leverage large, labeled datasets in order to learn generalizable features that can subsequently be transferred to unseen, related tasks, enabling more efficient learning from limited labeled data.

2.1.1. Transfer learning

The main idea of transfer learning is the transfer of knowledge from one task to another related task. This involves two phases. In the first phase, the ML model is trained on a (large) dataset associated with the first task (pre-training). In the second phase, the pre-trained model is taken as an initialization for training on a (smaller) dataset associated with the second task (fine-tuning).

Transfer learning methods have been widely used to address the imbalances in agricultural data available in different parts of the world. For example, models to perform crop yield estimation demonstrated that extrapolating between different countries in South America showed promising results (Wang et al., 2018). Kerner et al. (2020) generated a cropland map for Togo in less than ten days by developing rapid transfer learning techniques for regions with little to no available ground truth data. Hao et al. (2020) trained a random forest-based crop-type classification model on spatio-temporal Sentinel-2 data from areas in the United States and subsequently tested on regions in China and Canada. In general, transfer learning models showed promising capabilities in identifying crops, sometimes even without local training samples. Furthermore, it was noted that transfer learning techniques typically required longer time-series data compared to instances where sufficient local data was utilized for crop-type classification. Similarly, Antonijević et al. (2023) trained models to classify nine different types of crops in Brittany or Serbia while fine-tuning and testing them on data from the other respective area. They discovered that the transfer of knowledge from the original domain to the new was effective, both in terms of accuracy and computational efficiency.

2.1.2. Meta-learning

While transfer learning focuses on transferring knowledge and insights gained from one task to another related task, the core concept of meta-learning can be described as *learning-to-learn*.

In their seminal work, Thrun and Pratt (1998) introduce the concept of meta-learning as the task of learning properties of classes of functions from a few examples, that is, learning entire function spaces. On a less abstract level, the objective of meta-learning is to swiftly grasp a new task, a paradigm commonly referred to as *few-shot learning*. This typically involves adapting a machine learning model across several related tasks while monitoring the respective learning processes. The goal is then to improve the learning process itself to make a meta-learned model that is well suited for efficient adaptations to future

tasks (Rusu et al., 2018; Yoon et al., 2018; Lee et al., 2019; Nichol et al., 2018). For a contemporary overview, see also the survey of Hospedales et al. (2022).

Adapted to our crop classification problem, meta-learning aims to learn about the process of crop classification itself so that it can generalize to new tasks, such as new regions or unseen crops, with minimal effort. On the other hand, transfer learning uses a pre-trained model and tries to adapt it to classify crops from an unknown region.

Meta-learning approaches can be divided into three groups:

Model-based These methods are based on fully trained cyclic or recurrent neural network models with internal or external memory that learn to adapt to their state by reading a short sequence of task-specific training data. Examples include *Memory-Augmented Neural Networks* (Santoro et al., 2016) and *Neural Attentive Meta-Learners* (Mishra et al., 2018).

Metric-based These approaches learn effective and task-adapted distance metrics that are combined with non-parametric techniques during inference. Examples of this include *Prototypical Networks* (Snell et al., 2017), *Matching Networks* (Vinyals et al., 2016), and *Relation Networks* (Sung et al., 2018).

Optimization-based These algorithms represent a major research direction within the area of meta-learning, focusing on the optimization process of training ML models across different tasks. By gaining insights into these learning processes, they enable a joint optimization of, *e.g.*, model hyperparameters and parameter initializations across tasks, allowing for a rapid adaptation to new tasks. The most prominent example is MAML.

In this work, we will focus on the third category, as it is most widely applicable and fairly independent of the chosen ML architecture.

2.1.3. Meta-learning in remote sensing

Remote sensing encompasses a wide range of problems, such as semantic segmentation (*e.g.*, spatially resolved classifications of land cover types), change detection, and object detection. These often share underlying characteristic structures, making them suitable for meta-learning approaches (Schmitt et al., 2021). In fact, meta-learning has been successfully applied to geospatial data (Wang et al., 2020; Rußwurm et al., 2020b; Tseng et al., 2021, 2022).

A distinctive feature of remote sensing and Earth observation applications is the association of most data with specific geographic locations. These, in turn, are inherently tied to the prevalence of predictive quantities, such as crop type categories. Hence, meta-learning methods can learn to take advantage of these prevalences by observing tasks from different locations.

More specifically, for crop-type classification, there are three main reasons why the problem is predestined for meta-learning algorithms.

Seasonal variations Crop-type classification often involves monitoring crop growth and phenology over time. Meta-learning approaches can learn to adapt to temporal variations in crop appearance and spectral characteristics (Wang et al., 2020), allowing a more robust classification across different growing seasons and agricultural cycles.

Source regions Remote sensing data is collected from various geographical regions. The dimension of the source region captures the variability in environmental conditions, terrain types, climate patterns, and land use dynamics. Classification models may need to adapt to differences in data distribution between regions, such as rice fields on steep mountain terraces versus flat wheat fields, or mixed cropping systems versus large-scale monoculture farming.

Sensor modalities Satellite data can be captured using different sensors, each with its own spectral and temporal resolution, as well as other characteristics (Sainte Fare Garnot et al., 2020). Classification models may need to generalize across different sensor modalities while accounting for differences in data characteristics and noise patterns.

Explicitly addressing the second point, Tseng et al. (2022) introduced the *Task-Informed Meta-Learning (TIML; Tseng et al., 2022)* algorithm to study crop-type classification and yield estimation. It is a modification of the MAML algorithm (Finn et al., 2017) that incorporates prior location-dependent task information into the learned ML model. Tseng et al. (2022) found that the TIML algorithm performed best on average compared to four baseline methods on crop-type classification tasks in three different countries (Brazil, Kenya, and Togo) based on the CropHarvest dataset (Tseng et al., 2021). In particular, TIML outperformed MAML in all the regions considered. The baselines considered by Tseng et al. (2022) were a randomly initialized classifier, a classifier pre-trained on the binary CropHarvest crop-*vs.* -non-crop identification task (*cf.* Section 3), a Random Forest model, and the MAML algorithm.

In another study, Rußwurm et al. (2020b) observed that meta-learning algorithms can outperform regular pre-training of models on crop type data in a transfer learning setting when the dataset includes a distinct regional diversity. In the same work, they also found that meta-learning outperformed standard pre-training and randomly initialized baseline models without any pre-training when predicting crop types from heterogeneous sparse labels.

2.2. Self-supervised learning

To overcome the reliance on labeled pre-training data for crop-type classification, SSL has been explored as an alternative to the aforementioned supervised methods.

One notable approach is *Cropformer* (Wang et al., 2023), a two-step classification model that first undergoes self-supervised pre-training to capture crop growth patterns, followed by supervised fine-tuning. It demonstrated improvements in accuracy across various scenarios, including full-season and in-season crop classification, as well as regional transferability.

Wang et al. (2022) introduced *CC-SSL*, an end-to-end SSL framework for crop classification based on the novel *Sim-SCAN* contrastive learning algorithm. By incorporating a domain-specific input tensor transformation module capturing unique spatio-temporal crop growth patterns and sample balancing strategies, they achieved superior results compared to conventional supervised learning.

Senaras et al. (2023) focused on domain adaptation in crop classification, showing SSL's robustness to environmental and management variability across time and space in Germany.

MAEs for SSL were studied by Yuan et al. (2022), Cong et al. (2022) and Fuller et al. (2023) in the context of remote sensing. The *Contrastive Radar-Optical Masked Autoencoders (CROMA; Fuller et al., 2023)* algorithm processes both Sentinel-1 and Sentinel-2 data but only considers single observation data without temporal aspects. First SSL time-series models based on *Vision Transformer (ViT)* architectures turned out to be computationally expensive (Yuan and Lin, 2021). This was addressed by Tseng et al. (2023), resulting in PRESTO, which serves as a comparison baseline for our work. Its architecture is described in Section 4.3.3.

These methods highlight the effectiveness of SSL in leveraging unlabeled data for crop-type classification, particularly under limited label availability and across diverse environmental and agricultural conditions. However, they also underscore key limitations—most notably, high computational demands.

Building on these insights, we explore SSL as a scalable alternative for scenarios where transfer or meta-learning approaches are

constrained by a limited availability of labeled pre-training data. SSL can leverage large amounts of unlabeled data to extract generalizable features, potentially improving performance for tasks where labeled data are scarce or entirely absent.

2.3. Crop datasets

Significant obstacles persist in the development of ML platforms that work with agricultural Earth observation data. This is primarily due to different labeling and classification standards, biases in regional coverage, and the scarcity of publicly available labeled data.

The novel dataset EuroCropsML (Reuss and Macdonald, 2024; Reuss et al., 2025b), based on EuroCrops (Schneider et al., 2023b,a), overcomes this major hurdle by including international data that extends beyond political borders. This contrasts with national or regional datasets, such as the BreizhCrops dataset for Brittany (France) (Rußwurm et al., 2020a) and the ZueriCrop dataset for the cantons Thurgau and Zurich (Switzerland) (Turkoglu et al., 2021), which provide detailed information only for very local areas.

Although there also exists the Cropland Data Layer, a crop-type classification dataset published by the United States Department of Agriculture (Service, 2024), the labels provided here only have an accuracy slightly above 80 %, directly affecting the usability for ML methods. This is not the case for the EuroCropsML dataset, as it is based on the EuroCrops labeling, which contains accurate data directly self-reported by the farmers themselves.

Finally, the CropHarvest collection (Tseng et al., 2021) is a global dataset consisting of 90 480 data points that unify 20 different smaller datasets containing crop-type classification labels. The data is composed of Sentinel-2 top-of-atmosphere (L1C) observations, Sentinel-1 observations, ERA5 climatology data, and topography data from a Digital Elevation Model (Tseng et al., 2021). However, most of those data points merely distinguish between crop *vs.* non-crop labels. Only 34.2 % of the data points contain additional agricultural class labels, such as, *e.g.*, crop types. Furthermore, the types of crops in CropHarvest are generally grouped into only nine different classes (*cf.* Tseng et al., 2021, Figure 2b), namely `cereals`, `vegetables and melons`, `fruits and nuts`, `oilseed crops`, `root/tuber crops`, `beverage and spice crops`, `leguminous crops`, `sugar crops`, `other crops`. This is in stark contrast to the granular resolution of the hierarchical and harmonized labeling scheme offered by EuroCrops (*cf.* Table 1). Moreover, each CropHarvest data point contains a yearly time series at monthly intervals, which results in a much lower temporal resolution compared to the EuroCropsML dataset with up to 216 time steps (*cf.* Section 1).

The diverse geographical coverage, the high number of data points and time steps, as well as the harmonized crop class scheme, were the determining factors in our selection of EuroCropsML as the dataset for our benchmarking experiments of real-world crop-type classification.

3. Datasets

We rely on two distinct datasets for our analysis: EuroCropsML serves as the labeled reference dataset for supervised pre-training, fine-tuning, and evaluating classification performance, while unlabeled Pixel-Sen12MS-CR-TS data is utilized during the SSL pre-training phase to learn foundational data representations prior to task-specific fine-tuning. Our analysis focuses primarily on Sentinel-2 satellite data. However, in order to demonstrate the adaptability of the EuroCropsML dataset and its potential for integration with diverse data sources, we complement it with additional Sentinel-1 and ERA5 climate variables such as the daily average ground temperature (at 2 m height) and total daily precipitation. Together, Sentinel-1 and Sentinel-2 provide a rich, multi-modal and time-resolved view of the Earth's surface, combining structural and spectral information to improve monitoring and classification tasks in agriculture and beyond. ERA5 climate variables complement the satellite imagery by providing information that directly influences crop growth and phenology.

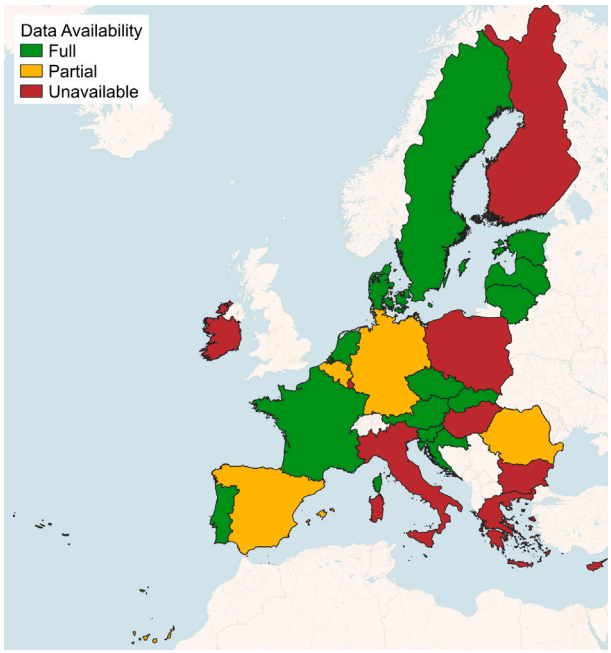


Fig. 2. Currently, 17 of the 27 member states of the European Union are harmonized within the EuroCrops dataset, with four states only offering partial data. The data availability refers to version 10 of the EuroCrops dataset (Schneider et al., 2023a).

3.1. The EuroCrops dataset

EuroCrops is a reference dataset, introduced by Schneider et al. (2023b), containing pan-European agricultural parcel information data and crop types. Data have been collected directly from farmers' self-declarations in accordance with the CAP of the EU. Hence, it provides a large collection of reliable, diverse, and unbiased reference data to develop and benchmark new ML methods and allows for in-depth investigation of agricultural activities.

Fig. 2 shows an overview of the countries that comprise the full EuroCrops dataset. Table 1 provides a more detailed account of these data, showing the availability of data together with the number of crop classes, identified parcels, and available years of coverage.

To overcome country-specific class names and taxonomies for crop types, the authors of the EuroCrops dataset introduced the so-called Hierarchical Crop and Agriculture Taxonomy (HCAT; Schneider et al., 2021, 2023b) to harmonize pan-European standards. This not only facilitates cross-regional transfer but also enables analyses of model performance at varying crop-type levels. HCAT level 3, *e.g.*, represents general land cover types, while level 4 defines the overall crop class, and level 5 the final crop type. If applicable, level 6 adds the seasonality, dividing specific crops into winter and spring.

3.2. The EuroCropsML benchmarking dataset

EuroCropsML (Reuss et al., 2025b) is a time-series dataset that combines parcel information reference data and multi-class HCAT labels from EuroCrops with Sentinel-2 L1C optical satellite observations captured during the year 2021. Each parcel data point in EuroCropsML contains an annual time series of cloud-free multi-spectral Sentinel-2 observation data, where each individual time step represents the median pixel value in the geometry of the parcel for each of the 13 Sentinel-2 spectral bands. In addition, the dataset contains further meta-data, including the spatial coordinates of the parcel's centroid and Eurostat's *GISCO Nomenclature of Territorial Units for Statistics (NUTS)* regions (Reuss et al., 2025b).

Table 1
Member states of the EU, sorted by the availability of their agricultural reference data. This refers to version 10 of the EuroCrops dataset (Schneider et al., 2023a). The *# crop classes* column refers to the number of unique crop types after HCAT harmonization. The *# parcels* column refers to unique parcel geometries after removing potential duplicates. EuroCropsML countries are marked bold.

countries	# crop classes	# parcels	years
Austria	98	2 610 470	2021
Croatia	11	1 377 155	2020
Czechia	142	665 679	2023
Denmark	127	587 453	2019
Estonia	127	176 055	2021
France	149	9 517 878	2018
Latvia	103	431 174	2021
Lithuania	22	1 102 441	2021
Netherlands	141	766 645	2020
Portugal	79	100 000	2021
Slovakia	118	255 568	2021
Slovenia	99	824 449	2021
Sweden	48	1 210 533	2021
Belgium[a]	129	591 083	2021
Flanders	129	591 083	2021
Germany[a]	205	1 921 513	2021, 2023
Brandenburg	116	286 246	2023
Lower Saxony	150	902 441	2021
NRW	175	732 826	2021
Romania[a]	8	329 706	N/A
Border region	8	329 706	
Spain[a]	10	996 679	2021
Navarra	10	996 679	2021

[a] The data for Belgium, Germany, Romania, and Spain is incomplete, meaning only data for selected regions are available. No data is yet available for Bulgaria, Cyprus, Finland, Greece, Hungary, Ireland, Italy, Luxembourg, Malta, and Poland.

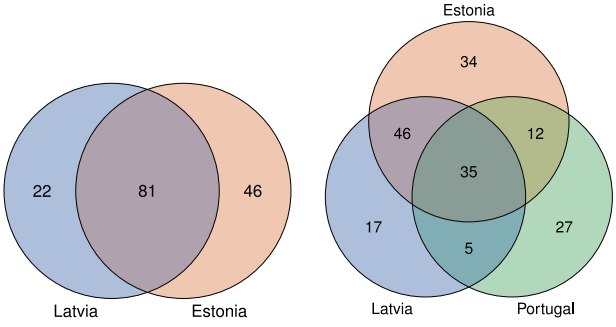


Fig. 3. The number of annotated crop classes that are shared and distinct between the three ROIs: Estonia, Latvia, and Portugal (Reuss et al., 2025b).

Tailored specifically for evaluating (few-shot) crop-type classification methodologies, this dataset encompasses agricultural data from three carefully selected European countries, providing a diverse and comprehensive resource for benchmarking ML algorithms within the agricultural domain.

3.2.1. Study area

The EuroCropsML dataset provides data for three European countries as *regions of interest (ROIs)*: **Estonia**, **Latvia**, and **Portugal**, as shown in Fig. 1.

In the context of supervised learning, Estonia and Latvia offer a suitable setting for investigating how similarities between neighboring countries affect knowledge transfer. Moreover, incorporating data from Portugal enables an examination of how differences in climate conditions, cultivation practices, and crop class distributions influence the transfer of knowledge from one region to a geographically distant one. This becomes particularly insightful when the geographic location of parcels is explicitly incorporated in ML models and algorithms — such as TIML — since the Baltic states and Portugal, despite being geographically distance, share several crop class occurrences, as illustrated in Fig. 3.

Table 2
The number of data points and distinct crop classes for the three countries that comprise the EuroCropsML dataset. The number of data points refers to the number of unique parcels after preprocessing.

Country	# data points	# crop classes
Estonia	175 906	127
Latvia	431 143	103
Portugal	99 634	79

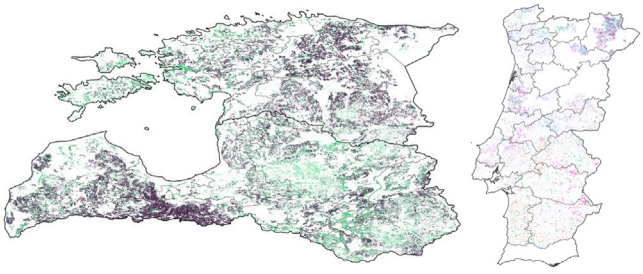


Fig. 4. Illustration of the distribution of crop-type labels for Estonia and Latvia (left) and Portugal (right), using EuroCrops HCAT3 level 3. The differences in the density of available data are clearly visible.

The details of the dataset are shown in Table 2. It consists of a total of 706 683 multi-class labeled data points with a total of 176 distinct classes, 35 of which are common to the three ROI countries, *cf.* Fig. 3.

The map shown in Fig. 4 visualizes the spatial distribution of crop classes for Estonia, Latvia, and Portugal, clearly unveiling significant

differences. For further illustrations of the different cultivation practices within the EuroCropsML countries, see Reuss et al. (2025b, Figs. 1–2).

Within our benchmark study, we fine-tune each model on the EuroCropsML Estonia data. While the SSL methods are pre-trained on unlabeled remote sensing data (*cf.* Section 3.3), we consider the following two scenarios for transfer and meta-learning (Reuss et al., 2025b):

Latvia→Estonia (LV→EE) The models are pre-trained on Latvian data and subsequently fine-tuned and evaluated on Estonian data. The pre-training dataset contains 103 distinct classes. During fine-tuning, the models see 127 classes in total, out of which 46 were not previously seen during pre-training.

Latvia+Portugal→Estonia (LV+PT→EE) The pre-training is conducted on data from Latvia and Portugal with a total of 142 classes, followed by fine-tuning and evaluating on data from Estonia. The fine-tuning dataset contains 127 distinct classes, of which 34 were not previously seen during pre-training.

3.2.2. Crop classes

The original composition of the dataset, the abundance of different crop classes, and the distribution of crop classes seen and unseen during the fine-tuning stage are illustrated in Fig. 5. The `pasture meadow grassland grass` class is the predominant class in Latvia (215 026 of 431 143 parcels) and Estonia (84 104 of 175 906 parcels) and the third most frequent class in Portugal (16 475 of 99 634 parcels). Hence, for the supervised pre-training stage, we resample it to the median frequency of all other classes. Fig. 5 (**bottom**) shows that the number of fine-tuning classes that were previously not seen during pre-training decreases when data from Portugal is added.

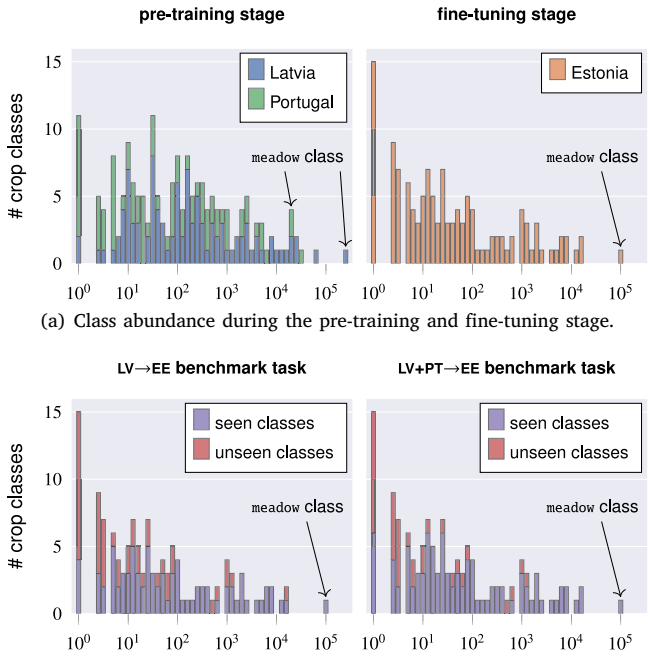
3.2.3. Splits

For the supervised pre-training stage, the pre-built data splits (Reuss et al., 2025b) allocate 80 % of the data to training and reserve the remaining 20 % for validation and hyperparameter tuning. The first use case (**LV→EE**) yields 172 993 samples in the training subset and 43 249 samples in the validation subset. For the second use case (**LV+PT→EE**), the training subset consists of 239 538 data points, while 59 885 are used for validation.

For the fine-tuning stage, the predefined splits reserve 60 % of Estonia's data for training (105 543 samples) and 20% for validation and testing, respectively. From the complete validation set, only 1000 fixed final data points are sampled; whereas, for testing, all 35 182 samples are kept.

3.2.4. Dataset extension

To complement the optical Sentinel-2 data, we collect Sentinel-1 C-band *Synthetic Aperture Radar (SAR) Ground Range Detected (GRD)* data for 2021 via the *Google Earth Engine (GEE)*. The imagery is selected based on the available orbit type (either ascending or descending) for each location and date, favoring descending when both are available. We use the *Interferometric Wide (IW)* mode with a dual-polarization configuration: *vertical transmit & vertical receive (VV)* and *vertical transmit & horizontal receive (VH)*. Following the EuroCropsML processing of Sentinel-2 data (Reuss et al., 2025b), we compute the median pixel value of the sigma nought (σ^0) Sentinel-1 backscatter in decibels for every parcel and temporal observation for both `VV` and `VH` bands independently. Additionally, ERA5 temperature and precipitation data are collected for each parcel and date with available satellite input.



(a) Class abundance during the pre-training and fine-tuning stage.

(b) Class abundance of overlapping and non-overlapping classes between pre-training and fine-tuning.

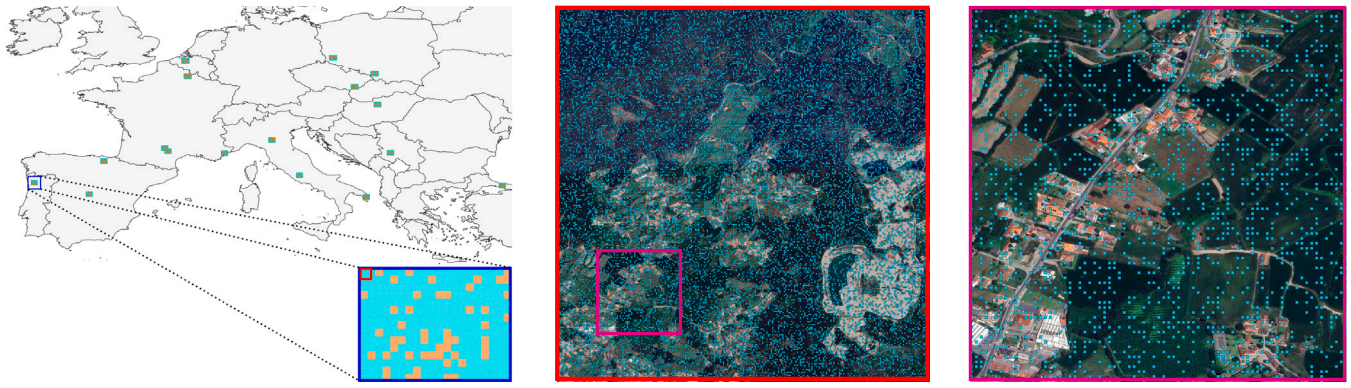
Fig. 5. EuroCropsML crop class abundances. (a) Histograms showing the (stacked) binned distribution of the number of crop classes of a certain abundance (# of parcels of that class) (in a log scale) in both the pre-training and fine-tuning datasets.
(b) Histograms showing the stacked binned distribution of the number of crop classes in the fine-tuning data from Estonia of a certain abundance (# of samples of that class) (in a log scale) that were previously seen or unseen during the supervised pre-training on either the data from Latvia only or data from Latvia and Portugal.

3.3. Pixel-SEN12MS-CR-TS

Pixel-Sen12MS-CR-TS is a comprehensive dataset tailored for pixel-wise SSL in remote sensing. It is derived from the European patches of the public Sen12MS-CR-TS dataset (Ebel et al., 2022), containing multi-temporal Sentinel-1 and Sentinel-2 imagery from 2018. All Sentinel-2 bands are upsampled to 10 m resolution. From the original 5520 256 × 256 px patches, we retain 5460 non-overlapping ones, from which we randomly allocate 80% to training and 20% for validation. For pixel-level sampling, 25% of the pixels per patch are randomly selected, yielding 89 456 640 samples in total, each with up to 60 time steps (30 from Sentinel-1 and Sentinel-2 each). Fig. 6 shows a visual representation of the dataset's patch and pixel distributions. We supplement the satellite data with ERA5 temperature and precipitation data, which we bilinearly interpolate to match the spatial resolution of the Sentinel data. In addition, we calculate the *Normalized Difference Vegetation Index (NDVI; Rouse et al., 1974)* from the Sentinel-2 bands B04 and B08, which is widely used as a general indicator for vegetation cover and health. A summary of the data structure similar to the one introduced by Tseng et al. (2023), *cf.* Section 4.3.3, is shown in Table 3. This dataset is used for pre-training our own PRESTO variants, *cf.* Section 5.

4. Methodology

In this section, we describe the ML models and the different learning algorithms that are examined in our benchmark study using the EuroCropsML dataset.



(a) PIXEL-SEN12MS-CR-TS train and validation patches.

(b) Random PIXEL-SEN12MS-CR-TS training patch from Portugal.

(c) Zoomed in area of the training patch from Portugal.

Fig. 6. PIXEL-SEN12MS-CR-TS dataset. (a) The map shows the distribution of training and validation patches in Europe. The blue square shows the spatial distribution of the Portugal patches. (b) A random training patch from Portugal (*cf.* red square in (a)), where each point represents a single sampled pixel. (c) The local pixel distribution of a randomly zoomed-in area of the training patch, shown as a magenta square in (b). Again, each sampled pixel is represented by a single point.

Table 3
Data structure of the PIXEL-SEN12MS-CR-TS dataset.

Variable type	Data source	Description
Dynamic	Sentinel-1	dual SAR VV and VH polarization channels
	Sentinel-2	B02 – B04 (visible); B05 – B07 (RE); B08 – B8A (NIR), B01, B09, B10 (atmospheric); B11 – B12 (SWIR)
	NDVI	computed from Sentinel-2 B04 (red) and B08 (NIR) bands
	ERA5	ERA5 reanalysis data with daily temperature and precipitation
Static	location	3D coordinates (x, y, z) from geographic latitude and longitude in Cartesian space

4.1. Transformer model

For all experimental settings, *cf.* Section 5, we rely on a state-of-the-art Transformer encoder architecture with sinusoidal positional encoding (Vaswani et al., 2017; Schneider and Körner, 2021). We set the maximum sequence length of input data to 366 days (*i.e.*, a full year, including one leap day). For supervised learning, this encoder model is combined with a single linear layer that maps Transformer encodings to the crop-type class logits. For all SSL methods, we combine the encoder with a transformer decoder. Further details of this architecture are outlined in Appendix A.1.

We refer to the encoder part as the *backbone* of the model and denote it as

$$M_{\text{backbone}}\left[\theta_{\text{backbone}}\right] : \mathbb{R}^{n_b \times n_t} \to \mathbb{R}^{n_e},$$

where $n_b \in \mathbb{N}$ and $n_t \in \mathbb{N}$ denote the number of bands (channels) and time steps in the multispectral temporal input data, respectively, and $n_e \in \mathbb{N}$ denotes the Transformer embedding dimension. All trainable model parameters of the backbone are contained in the parameter vector θ_{backbone}. Similarly, we refer to the classification layer during supervised learning and the SSL decoder as the *head* of the model. We denote it as

$$M_{\text{head}}\left[\theta_{\text{head}}\right] : \mathbb{R}^{n_e} \to \mathbb{R}^{n_c},$$

where, for supervised learning, $n_c \in \mathbb{N}$ denotes the number of distinct classes, and for SSL the Transformer decoder embedding dimension. As before, θ_{head} collects all trainable model parameters of the head. Note that while the model backbone architecture remains unchanged for all our experiments, the number of classes and, thus, the linear classification head changes between different pre-training, fine-tuning, and meta-learning tasks. Moreover, when fine-tuning a Transformer encoder–decoder architecture, we replace the decoder head with a linear classification layer (*cf.* Section 2.2).

The complete end-to-end model for a specific task is given by the composition

$$M[\theta] = M_{\text{head}}\left[\theta_{\text{head}}\right] \circ M_{\text{backbone}}\left[\theta_{\text{backbone}}\right],$$

where $\theta = \left[\theta_{\text{backbone}}, \theta_{\text{head}}\right]$ represents all trainable parameters from both the model backbone and head.

4.2. Supervised meta-learning

Meta-learning, as introduced by Thrun and Pratt (1998), see also the recent overview of Huisman et al. (2021), aims to implement the concept of *learning-to-learn*. In other words, meta-learning uses a variety of smaller related tasks to extract general information about a learning process. The goal is to obtain a meta-learned model that is able to quickly adapt to new and unknown scenarios and, thus, can learn an unseen task fast and efficiently.

In the following, we focus on optimization-based meta-learning methods, which can be formulated as bi-level optimization problems consisting of nested *inner* and *outer* optimization loops.

To illustrate the main ideas, consider a distribution $p(\mathcal{T})$ over a set of related tasks $\mathcal{T} = \{\tau^{(1)}, \tau^{(2)}, \ldots, \tau^{(N)}\}$, where each task $\tau^{(i)} = (D^{(i)}_{\text{support}}, D^{(i)}_{\text{query}}, \mathcal{L}^{(i)}) \in \mathcal{T}$ consists of a set of support (training) data points $D^{(i)}_{\text{support}}$, a set of query (test) data points $D^{(i)}_{\text{query}}$, and a loss function $\mathcal{L}^{(i)}$.

During *inner optimization*, we adapt a parametrized model $M[\theta]$ with learnable parameters θ on the support dataset by optimizing

$$\theta^{(i)}_{\text{opt}} = \mathcal{A}\left(M, D^{(i)}_{\text{support}}, \mathcal{L}^{(i)}, \omega\right)$$

$$\approx \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, \mathcal{L}^{(i)}\left(M\left[\boldsymbol{\theta}\right], \mathcal{D}^{(i)}_{\text{support}}\right),$$

where $\mathcal{A}$ is the chosen training algorithm, *e.g.*, mini-batch *stochastic gradient descent (SGD)*. This algorithm depends on hyperparameters $\boldsymbol{\omega}$, such as the learning rate or the initialization $\boldsymbol{\theta}_0$ of the model parameters.

The resulting model can be evaluated for the task $\tau^{(i)}$ on the query dataset according to the corresponding loss function $\mathcal{L}^{(i)}(M[\boldsymbol{\theta}^{(i)}_{\text{opt}}], \mathcal{D}^{(i)}_{\text{query}})$.

The goal of the *outer optimization* is then to improve these task evaluations by adapting the hyperparameters $\boldsymbol{\omega}$. The optimal values for all tasks are found by (approximately) identifying

$$\boldsymbol{\omega}_{\text{opt}} = \underset{\boldsymbol{\omega}}{\operatorname{argmin}}\, \mathbb{E}_{\tau^{(i)}\sim p(\mathcal{T})}\left[\mathcal{L}_{\text{meta}}\left(\mathcal{A}\left(M, \mathcal{D}^{(i)}_{\text{support}}, \mathcal{L}^{(i)}, \boldsymbol{\omega}\right), \mathcal{D}^{(i)}_{\text{query}}\right)\right],$$

where $\mathcal{L}_{\text{meta}}$ is some suitable meta-learning loss function.

One common meta-learning framework is *n-way k-shot classification* (Hospedales et al., 2022), which instantiates a specific variant of few-shot learning. In this case, each of the task training sets $\mathcal{D}^{(i)}_{\text{support}}$ consists of $n \cdot k$ data points from n distinct classes (k data samples per class) and the same classification loss $\mathcal{L}^{(i)}$, *e.g.*, the softmax cross-entropy loss, is used for all tasks.

In our experiments, *cf.* Section 5, we follow this n-way k-shot setup. We use a softmax cross-entropy classification loss for the inner optimization over the support sets. For the meta objective $\mathcal{L}_{\text{meta}}$, we use an averaged evaluation softmax cross-entropy classification loss on the query sets.

4.2.1. MAML

Building on this general concept, a widely adopted meta-learning approach is MAML. Specifically, it focuses on finding optimal initializations of the model that can quickly adapt to new tasks, measured by the number of training steps. In other words, the outer optimization over hyperparameters is restricted to the initialization of the model weights. Further, MAML relies on iterative gradient-based optimization methods, such as mini-batch *stochastic gradient descent (SGD)*, for both the inner and the outer optimizations. Here, the number of adaptation steps s for inner optimization is chosen as a fixed, typically rather small, constant. The respective step sizes (learning rates) for the two optimization iterations are consequently referred to as the inner and outer learning rates.

The interplay between these learning rates is complex, making it challenging to find suitable combinations in the MAML algorithm. Different approaches addressing this include the choice of a very fine-grained separate learning rate for each task in the inner optimization (Li et al., 2017), adaptive learning of inner and outer learning rates (Singh Behl et al., 2019), as well as the training of a separate model to generate per-layer learning rates (Baik et al., 2020).

4.2.2. FOMAML

Due to the nested structure of the MAML bi-level optimization, obtaining gradients for the outer optimization involves the computation of second-order derivatives with respect to the model parameters. This comes with significant computational costs.

Discarding higher-order derivatives by setting them equal to zero, we arrive at a variation of MAML with reduced computational costs. This is called *First-Order Model-Agnostic Meta-Learning (FOMAML; Finn et al., 2017)*.

4.2.3. ANIL

The *Almost No Inner Loop (ANIL; Raghu et al., 2019)* algorithm is another variation that streamlines the MAML algorithm in order to reduce its computational cost. This is achieved by restricting the inner optimization of MAML to only adapt the parameters $\boldsymbol{\theta}_{\text{head}}$ associated with the classification head of the model while keeping the backbone parameters $\boldsymbol{\theta}_{\text{backbone}}$ fixed.

This was motivated by the observations of Raghu et al. (2019) that for MAML, in many cases, the backbone parameters remained relatively stable across tasks. In their analysis for natural image classification tasks, the ANIL algorithm typically achieved a performance comparable to MAML.

4.2.4. TIML

The TIML algorithm represents an extension of MAML. It incorporates task-specific information, such as geospatial locations of a region or parcel, in the form of Cartesian coordinates. More precisely, given the geographic coordinates of a surface point $\boldsymbol{p}_{\text{polar}} = (\varphi, \lambda) \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \times [-\pi, \pi]$ represented by its spherical longitude φ and latitude λ, spatial information is encoded by mapping it into a normalized three-dimensional Cartesian coordinate space

$$\boldsymbol{p}_{\text{cart}} = (\cos(\lambda)\cdot\cos(\varphi), \cos(\lambda)\cdot\sin(\varphi), \sin(\lambda))^{\top} \in \mathbb{R}^3.$$

The original TIML algorithm only includes the spatial coordinates at the task level, more precisely, the central coordinates of the region or county associated with each task. Furthermore, Tseng et al. (2022) propose to add a one-hot encoded vector of ten higher-level crop categories to the task information. Thus, the task information remains constant within each individual task but changes across different tasks. We decided to make the following adjustments to TIML in our benchmark study:

1. We do not incorporate additional high-level information on crop categories since we replace the model head M_{head} to always exactly match the classes available in any task during each inner optimization. Hence, the model does not need more information on available crop categories.
2. We consider fine-grained spatial coordinates at the parcel level: We incorporate the coordinates of the centroid of each individual parcel present in a task instead of using the center of the county. The rationale behind this choice is that utilizing the location only at the county level can potentially result in spatial over-simplification. To illustrate, consider the case where a county exhibits an uneven distribution of crop classes, with certain crop-type clusters in the North and others in the South. Incorporating the same location information to all parcels would fail to resolve these distinct clusters.

This leaves us with a unique 3-dimensional task information vector for each data point. This is either directly concatenated with the remaining time series (no encoder) or encoded into specific layers of the model architecture (encoder); see also Tseng et al. (2022) for details. For the latter, these encodings are treated as learnable model parameters during outer optimization. In accordance with the approach outlined in Tseng et al. (2022), we apply them once prior to the model backbone and once prior to the model head. In contrast to the model architecture originally considered by Tseng et al. (2022), we use only one layer for the classification head, whereas they allow multiple layers and apply the task encoding before each of them.

Additionally, when using the task encoder, we pad all input sequences to 366 days and mask out the padded values since the encoder requires fixed-length input sequences. All remaining parameters can be obtained from Tseng et al. (2022).

Due to the less streamlined nature of the CropHarvest dataset considered by Tseng et al. (2022), the authors also employ a forgetful meta-learning routine. This is proposed to overcome the challenge that the majority of CropHarvest data points only allow for a binary classification into crop-*vs.*-non-crop categories, which creates a bias for meta-learning algorithms toward learning only this specific task. However, we skip the incorporation of forgetful meta-learning, since the nature of the EuroCropsML dataset already inherently provides room for extensive and diverse multi-class tasks.

4.3. Self-supervised learning

We compare various lightweight variations of MAEs in this study. As our baseline, we adopt the PRESTO model (Tseng et al., 2023), an efficient time-resolved MAE designed for remote sensing applications. Building on this, we introduce several modifications aimed at improving temporal and spectral resolution, as well as computational efficiency.

4.3.1. Masked autoencoders

MAEs are ViT-based models for SSL. They consist of an encoder–decoder architecture in which random patches of input images are masked out for the encoder, to later be reconstructed by the decoder. MAEs are trained using a regression loss, *e.g.*, the *mean squared error (MSE)*, matching decoder reconstructions against original unmasked inputs.

After SSL pre-training, MAEs can be used for various downstream tasks. For this, typically only the encoder part of the MAE is retained while the decoder part is dropped and replaced by a task-specific partial model. For our classification task, this means that after pre-training, we replace the initial decoder with a simple linear classification layer, which is subsequently fine-tuned to predict the class log-probabilities.

4.3.2. Cross-attention masked autoencoders

Fu et al. (2025) investigated the necessity of masked tokens interacting via self-attention during the reconstruction process in MAEs, versus solely relying on cross-attention between masked and unmasked tokens. They found that MAE decoders can learn strong global representations from unmasked inputs alone, making masked-to-masked interactions unnecessary. Consequently, they introduced *Cross-Attention Masked Autoencoder (CrossMAE; Fu et al., 2025)*, which replaces the decoder's self-attention with a simpler, computationally less expensive cross-attention mechanism. This allows the decoder's masked tokens (queries Q) to attend exclusively to visible tokens (keys K and values V) from the encoder's output.

4.3.3. The PRESTO model

Adapting the MAEs framework to pixel-wise time-resolved remote sensing data, PRESTO (Tseng et al., 2023) follows the encoder–decoder architecture with masking but processes individual pixel time-series data instead of static image patches. It leverages domain-specific features, such as temporal structure and multi-modal data, as found in datasets like Pixel-Sen12MS-CR-TS. Tseng et al. (2023) observed that PRESTO could match the performance of much larger models while being significantly more computationally efficient. The main strengths of PRESTO are:

Parameter efficiency: PRESTO achieves accuracy on par with large-scale models like ViT or *Residual Network (ResNet; He et al., 2016)*, while requiring up to 1000 times less learnable parameters.

Versatility across remote sensing sources: Trained on diverse inputs (*e.g.*, Sentinel-1 and Sentinel-2), PRESTO can handle data from multiple sensors concurrently.

Robustness to missing data: During the masked pre-training, PRESTO adapts to process incomplete input data, such as data with temporal or spatial gaps, making it more versatile for different downstream tasks.

The PRESTO model is pre-trained using the same combination of dynamic (time-resolved) data (Sentinel-1, Sentinel-2, NDVI, ERA5) and static data (location) as described in Section 3.3, with the addition of nine *Dynamic World Land Cover* classes (*i.e.*, `water`, `trees`, `grass`, `crops`, `shrub & scrub`, `flooded vegetation`, `built-up area`, `bare ground`, and `snow & ice`; *cf.* Brown et al., 2022) for each non-cloudy Sentinel-2 observation, and (static) *topography data*, specifically elevation and slope, derived from the *Shuttle Radar Topography Mission (SRTM; Rabus et al., 2003) Digital Elevation Model (DEM)*. Moreover, from the 13 Sentinel-2 bands, the bands `B01`, `B09`, and `B10` are removed and not processed by the PRESTO model.

Input encoding. In order to handle different variable types (categorical and numerical) and varying numbers of input channels, PRESTO introduces a specialized unified input encoding pipeline:

Channel grouping: The input variables are divided into C data source (*e.g.*, sensor) specific channel groups, with $\sum_{c=1}^{C} d_c = D$, where D is the total number of channels and d_c is the number of channels belonging to group c. All Sentinel-1 bands, *e.g.*, form one group. See the original work of Tseng et al. (2023) for the detailed description of the channel groups.

Channel group embedding: For each channel group c, the input data with d_c channels is passed through a learned linear projection $h^c : \mathbb{R}^{d_c} \to \mathbb{R}^{d_{\text{emb}}}$, mapping the group's channels into a shared latent embedding space of dimension d_{emb}. For dynamic channel groups, this transformation is applied independently to each time step. Categorical variables (such as land cover class) are implicitly one-hot-encoded, making the linear transformation equivalent to a learned embedding matrix.

Contextual encodings: Where applicable, additive encodings are used for token contextualization:

Temporal position encoding: A sinusoidal function $p_{\sin} : \{1, \dots, T\} \to \mathbb{R}^{d_{\sin}}$, where $d_{\sin} = \frac{1}{2} \cdot d_{\text{emb}}$, and T denotes the number of all time steps, encodes each token's temporal position in the time series, as common in Transformer models (Vaswani et al., 2017).

Month encoding: Similarly, a sinusoidal encoding $p_{\text{month}} : \{1, \dots, T\} \to \mathbb{R}^{d_{\text{month}}}$, with $d_{\text{month}} = \frac{1}{4} \cdot d_{\text{emb}}$, that only depends on the month of an observation's time step is intended to capture seasonal patterns.

Channel group encoding: Finally, a learned embedding $p_{\text{channel}} : \{1, \dots, C\} \to \mathbb{R}^{d_{\text{channel}}}$, with $d_{\text{channel}} = \frac{1}{4} \cdot d_{\text{emb}}$, indicates the channel group (*i.e.*, data source) of each token.

The full contextual embedding dimensions are chosen to sum up to the same shared embedding dimension, *i.e.*, $d_{\sin} + d_{\text{month}} + d_{\text{channel}} = d_{\text{emb}}$.

A full input example is constructed by combining the previously defined embeddings as follows: For a dynamic variable channel group c, the raw data t_i^c for a time step i is encoded as $h^c(t_i^c) + [p_{\text{channel}}(c); p_{\sin}(i); p_{\text{month}}(i)] \in \mathbb{R}^{d_{\text{emb}}}$. For static variable groups, the temporal encodings are set to zero, since these variables are independent of time, resulting correspondingly in $h^c(t^c) + [p_{\text{channel}}(c); \mathbf{0}; \mathbf{0}] \in \mathbb{R}^{d_{\text{emb}}}$. Finally, all embeddings for static variable groups and dynamic variable groups are concatenated to form the input $\mathbf{x} \in \mathbb{R}^{(C_{\text{static}} + C_{\text{dynamic}} \cdot T) \times d_{\text{emb}}}$, where C_{static} and C_{dynamic} denote the number of static and dynamic groups and together sum to the total number of groups $C_{\text{static}} + C_{\text{dynamic}} = C$.

Masking. Given the inherent challenges of remote sensing data, which is often partially missing information, *e.g.*, due to sensor noise or cloud coverage, PRESTO employs four structured masking strategies during pre-training aiming at masking a fraction of 75 % of input tokens:

Random: Masking random pairs of time steps and channels.

Channel groups: Masking a random subset of channel groups. In the case of dynamic channel groups, this will mask all time steps from the groups.

Contiguous time steps: Mask a random subset of consecutive time steps across all dynamic channels.

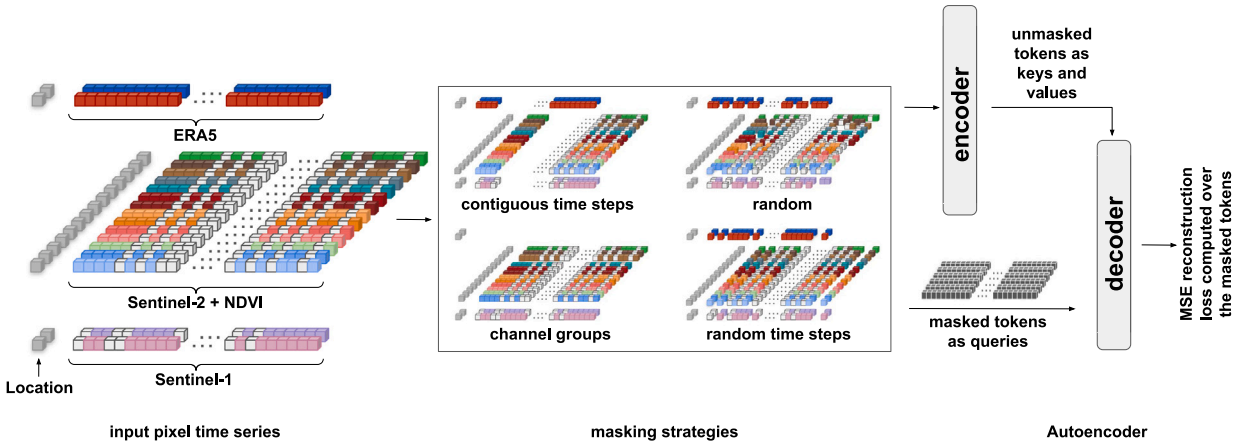


Fig. 7. Visualization of CrossPRESTO-XTS using the full pre-training dataset described in Section 3.3. The static location variable is appended as an additional time step to the dynamic variables, as proposed by Tseng et al. (2023). The unmasked tokens serve as inputs for the encoder, before its output is used as keys and values to perform cross-attention in the decoder with the masked tokens as queries. Finally, the loss is computed over the reconstructed masked tokens only. The white cubes in the data input represent padded time steps. These tokens are also masked out in the encoder but not taken into account during loss computation.

Random time steps: Mask a random subset of time steps across all dynamic channels.

In the context of channel group and time step masking, if not exactly 75 % of all tokens can be masked with these strategies, a slightly smaller fraction is selected first, and the remaining difference is randomly masked. Using a random combination of all four masking strategies in the encoder–decoder architecture, PRESTO is trained exactly as MAEs, using an MSE regression loss matching the decoder reconstructions against the original unmasked input.

4.3.4. Modifications to PRESTO

We propose the following variations to the original PRESTO model to maximize temporal and spectral flexibility for downstream tasks and better align it with our pre-train dataset PIXEL-SEN12MS-CR-TS (*cf.* Section 3.3) and experimental setup (Section 5). In particular, we make the following modifications:

Input encoding: We no longer restrict the model to 12 monthly time steps, and thus remove the month encoding p_{month} from the contextual embedding. Hence, the shared embedding dimension becomes $d_{\text{emb}} = d_{\text{sin}} + d_{\text{channel}}$, with $d_{\text{sin}} = \frac{3}{4} \cdot d_{\text{emb}}$ and $d_{\text{channel}} = \frac{1}{4} \cdot d_{\text{emb}}$.

Masking strategy: Unlike PRESTO, which uses random fallback masking to maintain a fixed ratio, we strictly adhere to the initial structured mask, even if fewer tokens are masked, with a masking ratio of 70 %.

Selection of spectral bands: To preserve the full spectral range of Sentinel-2, all bands (including bands B01, B09, and B10) are retained throughout. If needed, individual bands are masked during fine-tuning rather than removed, improving model versatility without imposing predefined constraints.

We refer to this version of the model as PRESTO-XTS.

While additional spectral bands and higher temporal resolution enhance representational capacity, they also increase computational demands. To mitigate this, we introduce CrossPRESTO-XTS — a second, computationally more efficient variant — by integrating the cross-attention mechanism from Section 4.3.2 into the PRESTO-XTS model. The final architecture is illustrated in Fig. 7.

5. Experimental setup

All models are trained end-to-end, following a standard pre-training and fine-tuning paradigm. In particular, all supervised methods are pre-trained on the EUROCROPSML pre-training dataset, before fine-tuning each model on the EUROCROPSML Estonia data. On the other hand, all SSL algorithms are pre-trained on PIXEL-SEN12MS-CR-TS, before also being fine-tuned on the Estonia data.

In addition, we consider training a randomly initialized Transformer encoder with a linear classifier from scratch without any pre-training on the same fine-tuning tasks as a baseline comparison. This is referred to as *no pre-training*.

Hyperparameters are optimized by computing the loss (during pre-training) or classification accuracy (during fine-tuning) on a held-out validation dataset. Appendix A.2 features a detailed outline of the hyperparameter tuning setup.

5.1. Supervised pre-training

We use all training data for the transfer learning baseline to pre-train the Transformer model and validate it on the validation data. This already involves a downsampling of the `pasture meadow grassland grass` class to the median frequency of all other classes, *cf.* Reuss et al. (2025b), allowing to learn better representations during pre-training. From the pre-processed EUROCROPSML data, we keep all 13 Sentinel-2 spectral bands except band B10 (cirrus SWIR), which is typically used for cloud detection. This band is not further necessary here since the data EUROCROPSML pre-processing pipeline (Reuss et al., 2025b) already performs cloud removal. In order to identify the optimal setting for transfer learning, we consider batch size and cosine annealing as additional hyperparameters to the learning rate. The detailed configuration can be found in Appendix A.4.

For meta-learning algorithms, we further sample meta-training and meta-validation tasks $\tau \in \mathcal{T}$ following the n-way k-shot paradigm for few-shot learning as follows:

Step 1: Sampling a NUTS-region With a probability given by the ratio of the number of data points per region to the total number of data points, we sample one of the regions LV003, LV005, LV006, LV007, LV008, LV009, PT11, PT15, PT16, PT17, PT18, where the Portuguese regions (PT) are only considered for the second use case (**LV+PT→EE**) mentioned above. For Latvia (LV) the regions correspond to the country's NUTS level-3 regions. For Portugal (PT) they reflect NUTS level-2 regions, as

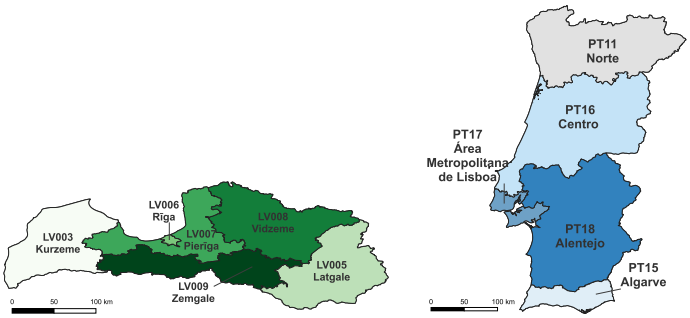


Fig. 8. NUTS regions for (a) Latvia and (b) Portugal used for sampling meta-learning tasks.

the Portuguese NUTS level-3 regions are relatively small and would hinder the creation of meaningful meta-learning tasks. A visualization of the NUTS regions can be seen in Fig. 8.

Step 2: Sampling a task We uniformly sample n random distinct classes that are present in the previously selected region. For each class, we then uniformly sample k_{query} random data points for the query set D_{query}. Similarly, from the remaining samples, for each of the n classes, we uniformly select k_{support} random data points for the support set D_{support} of the task. If, after sampling the query set, there are fewer than k_{support} data points left for sampling from a specific class, then we include all of them in the support set.

Experiments are conducted for $n \in \{4, 10\}$ and $k \in \{1, 10\}$. For all meta-learning algorithms and n-way k-shot settings, we consider the simplest algorithm variant with one gradient step (*i.e.*, $s = 1$) in the inner optimization. For the setting $n = 4$ and $k = 1$, we also consider variants with multiple inner gradient steps $s \in \{1, 4, 10\}$. A more detailed outline and analysis of the meta-learning settings can be found in Appendix A.3.

During meta-learning, we sample a fixed set of 100 meta-validation tasks for model validation and hyperparameter tuning, *cf.* Appendix A.2, from the pre-training validation subset following steps 1 and 2 so that model validation is always performed on the same set of tasks.

5.1.1. Self-supervised pre-training

For the baseline PRESTO model, we obtained the original publicly available model parameters pre-trained by Tseng et al. (2023). PRESTO-XTS and CrossPRESTO-XTS were pre-trained from scratch on all channels of the PASTIS-SEN12MS-CR-TS using MSE loss over the masked tokens only. Additional results using a larger Transformer architecture and deterministic masking strategies, *i.e.*, masking entire channel groups and contiguous time steps, are relegated to Appendix B.4.

5.2. Fine-tuning setup

In order to assess the ability of different pre-training algorithms to cope with data-scarce situations, we fine-tune the models in various few-shot settings with 1, 5, 10, 20, 100, 200, or 500 shots (*cf.* Reuss et al., 2025b). These few-shot learning tasks within EUROCROPSML are constructed by sampling from the already imbalanced training data of Estonia, shown in Fig. 5(a). Due to this inherent class imbalance, the maximum number of samples that can realistically be drawn per class is strictly constrained by the underlying label distribution. Consequently, as the required number of training samples (shots) increases, the sampled prior distribution for training effectively converges toward the true empirical prior for the entire Estonian dataset. Moreover, since our goal is to benchmark few-shot learning in real-world settings, the test set of Estonia retains the empirical prior distribution and remains highly imbalanced. Hence, the few-shot settings presented in this work test the methods' generalization performance regarding both a balanced and empirical underlying training label distributions.

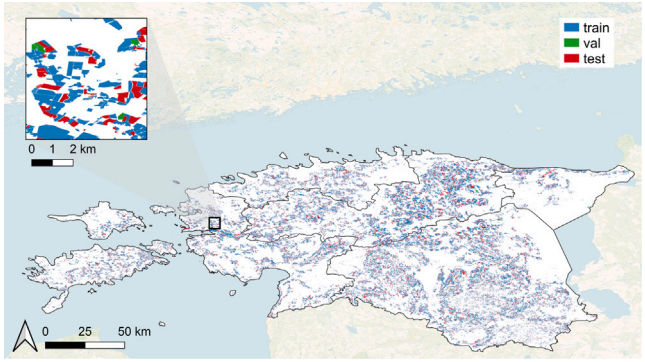


Fig. 9. Spatial distribution of the fine-tuning dataset. The entire Estonia data is randomly split into train (60 %), validation (20 %), and test (20 %). For the final validation set, only 1000 samples are used. As shown on the map, the data distribution is spatially uniform throughout the country.

We train and evaluate the models on the final fine-grained HCAT3 level 6 classes (Schneider et al., 2021). A performance analysis on broader categories, as *e.g.*, land cover classes, is deferred to Appendix B.2.

Classification performance during fine-tuning is evaluated using *overall classification accuracy* a_{OA} (micro-averaged across instances from all classes). To assess minority-class performance, we also report the accuracy on the fine-tuning sets excluding the `pasture meadow grassland grass` majority class (a_{MCA}). Additional analyses, including hyperparameter choices and evaluations with *Cohen's kappa coefficient* κ metric, are deferred to Appendix B.

All models are fine-tuned for up to 200 epochs with a batch size of 16. We use the 1000 fixed data points sampled from the validation subset for model validation and hyperparameter tuning. Early stopping is triggered if the validation loss does not improve for five epochs. The final models are evaluated on all 35 182 test samples. We evaluate three learning rate strategies:

1. The same learning rate applies to the backbone model and the randomly initialized classification head.
2. Separate, independently optimized learning rates are used for the backbone and the classification head.
3. Only the classification head is fine-tuned with a tuned learning rate, while the backbone learning rate is fixed to zero, effectively freezing the backbone.

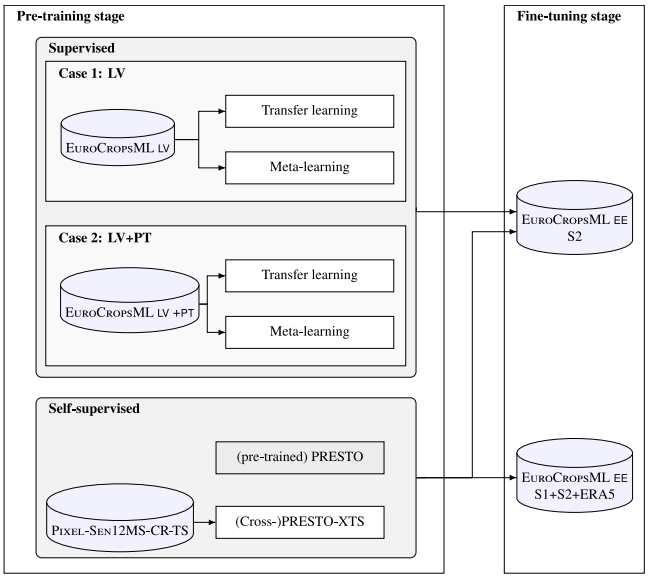


Fig. 10. Overview of the pre-training and fine-tuning setups. The supervised algorithms are either pre-trained first on EuroCropsML LV or EuroCropsML LV + PT, before being fine-tuned on EuroCropsML EE S2. While PRESTO-XTS and CrossPRESTO-XTS are pre-trained on Pixel-Sen12MS-CR-TS via reconstruction, for the original PRESTO, we use the pre-trained weights (Tseng et al., 2023). Hence, no additional pre-training is conducted with the PRESTO model. All SSL models are subsequently fine-tuned on either EuroCropsML EE S2 or EuroCropsML EE S1+S2+ERA5.

In order to evaluate the robustness of the results and reproducibility of the algorithms' performances, we conduct all experiments five times, repeated with different random seeds (influencing, *e.g.*, random parameter initialization and mini-batch sampling, *cf.* Appendix A.6).

In addition, we train the baseline on all of the training data from Estonia to quantify the complexity of the downstream task in a non-few-shot setting. Fig. 9 visualizes the spatial distribution of the fine-tuning dataset, including all training samples.

In order to evaluate model performance, we define two fine-tuning scenarios with different input modalities:

EuroCropsML S2: The fine-tuning data exclusively contains Sentinel-2 imagery (and NDVI in the case of SSL). For SSL, this reflects only a subset of the data available during pre-training.

EuroCropsML S1+S2+ERA5: The fine-tuning data contains additional data modalities, such as Sentinel-1 and ERA5 data. This scenario is only evaluated for SSL and demonstrates benchmarking on an extended version of the EuroCropsML dataset. For PRESTO-XTS and CrossPRESTO-XTS, this reflects a setup, where all input modalities from pre-training (Section 3.3) are present during fine-tuning.

Again, in terms of spectral resolution, in general, we keep all 13 Sentinel-2 bands except band B10. For fine-tuning the PRESTO model, we also remove bands B01 and B09, as done in Tseng et al. (2023), *cf.* Section 4.3.3.

Fig. 10 gives an overview of the full training processes.

6. Results

We present the best results for each algorithm and fine-tuning task in this section and defer a detailed analysis of algorithm variations and hyperparameter influences to Appendices A.2 amd B. The tables in this section compare the randomly initialized baseline model (no pre-training) to transfer learning, the meta-learning algorithms ANIL, MAML, FOMAML, and TIML (with and without encoder), and variants of the SSL model PRESTO. In each case, we report the mean and standard deviation of a_{OA} estimated from the repeated experiments with varying random seeds, as detailed in Section 5.2 and Appendix A.6.

6.1. Results on EuroCropsML S2

When training the randomly initialized baseline model on the full training data of the **EuroCropsML S2** fine-tuning task, we receive a final accuracy of $a_{\text{OA}} = 0.787 \pm 0.004$ on the test set. This can serve as a general reference value for the overall difficulty of the learning task on the Estonian data.

Table 4a highlights the results when fine-tuning the pre-trained models on the data from Estonia. Tables 4c and 4b provide additional insights into a more comprehensive analysis of the impact of the `pasture meadow grassland grass` class and minority-class performance.

6.1.1. Supervised learning

Table 4a shows the overall accuracy for each algorithm and task. For **LV→EE**, the standard transfer learning approach did not lead to any improvement in precision, compared to training from scratch. Algorithms belonging to the MAML family (*i.e.*, ANIL, (FO-)MAML) tend to perform better as the number of fine-tuning shots increases, reaching a maximum of $a_{\text{OA}} \approx 65\,\%$ for the 500-shot benchmark task with MAML. Furthermore, TIML (without an encoder) shows comparable results. The encoder version of TIML achieves the highest classification accuracy for the extremely data-scarce 1-shot benchmark task, outperforming alternative approaches by a significant margin. However, this approach exhibits suboptimal performance for tasks with 100 or more shots, in comparison to the remaining **LV→EE** algorithms.

Compared to pre-training on LV only, the incorporation of additional data from Portugal during pre-training (**LV+PT→EE**) did not result in general improvements during fine-tuning. This becomes quite evident when examining the baseline model that has not been exposed to any data from Portugal. It outperforms all supervised **LV+PT→EE** algorithms for the 1-, 5-, 10- and 20-shot benchmark tasks.

Surprisingly, pre-training on Portuguese data often boosts performance on minority classes (Table 4b). While the majority of methods show a drop in accuracy, models exposed to Portugal data in 1- and 5-shot settings achieve even higher minority-class accuracies than overall accuracies (*cf.* Table 4a). Most notably, **LV+PT→EE** attains the highest scores for 1-, 5-, 100-, and 200-shots through various meta-learning algorithms.

As might be reasonably anticipated, the results displayed in Table 4c show a contrary behavior. While the no-pretraining baseline leads the 5- and 10-shot tasks, pre-training on Latvian data yields the highest performances, with MAML winning from 100 shots onward. Regular transfer learning, while achieving strong results on minority-classes, is encountering difficulties when being evaluated exclusively on the meadow class.

Table 5a (**LV→EE**) and Table 5b (**LV+PT→EE**) enhance the understanding through a more comprehensive analysis of shared and non-shared class subsets between pre-training and fine-tuning countries. In particular, they demonstrate the dominance of the `pasture meadow grassland grass` class in the learned data.[1] For the overlapping classes (EE ∩ LV) in the learning task **LV→EE** (*cf.* Table 5a), the MAML algorithm demonstrates the best performance, attaining an accuracy of approximately 56%. In the context of the learning task pertaining to Portuguese data, as indicated in Table 5b, the accuracy

[1] The selected 20-shot scenario serves as a representative case, as presenting these results for all k-shot settings would exceed the scope of this section.

Table 4
Classification accuracy for the **EuroCropsML S2** benchmark. We report the classification accuracy on the test set for each algorithm and few-shot task. Test metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per few-shot scenario is marked in **blue bold**. Black bold values highlight the best result for supervised or SSL pre-training. Additionally, cases in which the **LV+PT→EE** was an improvement over the corresponding **LV→EE** result are highlighted in green color.

(a) Fine-tuning classification results for the full **EuroCropsML S2** task.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.17 ± 0.14	**0.33 ± 0.11**	**0.438 ± 0.027**	0.490 ± 0.020	0.518 ± 0.033	0.573 ± 0.019	0.6317 ± 0.0062
	transfer learning (LV)	0.143 ± 0.070	0.203 ± 0.081	0.254 ± 0.060	0.367 ± 0.087	0.465 ± 0.043	0.503 ± 0.032	0.575 ± 0.048
	ANIL (LV)	0.189 ± 0.047	0.250 ± 0.076	**0.419 ± 0.045**	0.490 ± 0.069	**0.561 ± 0.034**	0.589 ± 0.020	**0.652 ± 0.011**
	MAML (LV)	0.176 ± 0.077	**0.259 ± 0.088**	0.390 ± 0.035	**0.507 ± 0.057**	0.552 ± 0.023	**0.611 ± 0.043**	0.648 ± 0.017
	FOMAML (LV)	0.21 ± 0.11	0.228 ± 0.052	0.408 ± 0.065	0.453 ± 0.080	0.516 ± 0.036	0.557 ± 0.018	0.6191 ± 0.0069
	TIML (encoder) (LV)	**0.247 ± 0.092**	0.25 ± 0.11	0.368 ± 0.040	0.423 ± 0.067	0.494 ± 0.045	0.517 ± 0.022	0.591 ± 0.021
	TIML (no encoder) (LV)	0.18 ± 0.12	0.26 ± 0.10	0.337 ± 0.069	0.456 ± 0.058	0.530 ± 0.044	0.535 ± 0.018	0.637 ± 0.010
	transfer learning (LV + PT)	0.0787 ± 0.0091	0.1233 ± 0.0058	0.249 ± 0.098	0.292 ± 0.089	0.405 ± 0.023	0.473 ± 0.033	0.583 ± 0.028
	ANIL (LV + PT)	0.154 ± 0.050	0.200 ± 0.020	0.321 ± 0.090	0.480 ± 0.045	0.538 ± 0.028	0.5761 ± 0.0098	0.623 ± 0.019
	MAML (LV + PT)	0.117 ± 0.013	0.183 ± 0.018	0.366 ± 0.086	0.452 ± 0.094	0.517 ± 0.016	0.568 ± 0.017	0.637 ± 0.014
	FOMAML (LV + PT)	0.103 ± 0.011	0.192 ± 0.035	0.33 ± 0.10	0.445 ± 0.058	0.512 ± 0.014	0.562 ± 0.028	0.623 ± 0.011
	TIML (encoder) (LV + PT)	0.128 ± 0.031	0.163 ± 0.031	0.344 ± 0.084	0.438 ± 0.031	0.488 ± 0.029	0.473 ± 0.038	0.587 ± 0.024
	TIML (no encoder) (LV + PT)	0.162 ± 0.074	0.189 ± 0.027	0.299 ± 0.018	0.441 ± 0.070	0.516 ± 0.063	0.538 ± 0.021	0.622 ± 0.012
	PRESTO	**0.157 ± 0.078**	0.24 ± 0.12	0.236 ± 0.030	0.322 ± 0.064	0.458 ± 0.045	**0.491 ± 0.055**	0.5528 ± 0.0037
	PRESTO-XTS	0.050 ± 0.025	0.18 ± 0.15	0.276 ± 0.094	0.443 ± 0.024	**0.501 ± 0.035**	0.486 ± 0.017	**0.622 ± 0.021**
	CrossPRESTO-XTS	0.09 ± 0.10	**0.32 ± 0.18**	**0.300 ± 0.092**	**0.474 ± 0.011**	0.495 ± 0.018	0.468 ± 0.022	0.599 ± 0.018

(b) Fine-tuning classification results for **EuroCropsML S2** task excluding the majority class `pasture meadow grassland grass`.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.050 ± 0.045	0.037 ± 0.038	0.060 ± 0.045	0.142 ± 0.097	0.27 ± 0.11	0.455 ± 0.027	0.517 ± 0.023
	transfer learning (LV)	0.15 ± 0.11	0.20 ± 0.13	0.24 ± 0.17	0.295 ± 0.095	0.389 ± 0.098	0.431 ± 0.072	0.531 ± 0.058
	ANIL (LV)	0.10 ± 0.13	0.20 ± 0.11	0.18 ± 0.12	**0.308 ± 0.052**	0.355 ± 0.088	0.451 ± 0.039	**0.5478 ± 0.0096**
	MAML (LV)	0.14 ± 0.11	0.065 ± 0.086	**0.263 ± 0.045**	0.282 ± 0.060	0.300 ± 0.056	0.385 ± 0.070	0.480 ± 0.083
	FOMAML (LV)	0.08 ± 0.11	0.088 ± 0.099	0.074 ± 0.080	0.170 ± 0.081	0.324 ± 0.093	0.440 ± 0.026	0.486 ± 0.023
	TIML (encoder) (LV)	0.042 ± 0.032	0.117 ± 0.051	0.067 ± 0.058	0.153 ± 0.036	0.299 ± 0.055	0.289 ± 0.070	0.450 ± 0.031
	TIML (no encoder) (LV)	0.11 ± 0.10	0.209 ± 0.092	0.206 ± 0.085	0.261 ± 0.090	0.377 ± 0.053	0.421 ± 0.062	0.546 ± 0.019
	transfer learning (LV + PT)	0.144 ± 0.018	0.227 ± 0.020	0.202 ± 0.051	0.256 ± 0.041	0.328 ± 0.095	0.439 ± 0.037	0.489 ± 0.042
	ANIL (LV + PT)	0.165 ± 0.083	**0.272 ± 0.032**	0.223 ± 0.087	0.248 ± 0.063	0.325 ± 0.067	0.435 ± 0.071	0.501 ± 0.041
	MAML (LV + PT)	**0.191 ± 0.056**	0.230 ± 0.018	0.175 ± 0.087	0.263 ± 0.033	0.321 ± 0.078	0.443 ± 0.023	0.527 ± 0.017
	FOMAML (LV + PT)	0.159 ± 0.084	0.19 ± 0.11	0.140 ± 0.096	0.283 ± 0.076	0.336 ± 0.074	**0.458 ± 0.031**	0.513 ± 0.030
	TIML (encoder) (LV + PT)	0.107 ± 0.083	0.118 ± 0.078	0.111 ± 0.082	0.121 ± 0.069	0.244 ± 0.061	0.299 ± 0.069	0.404 ± 0.072
	TIML (no encoder) (LV + PT)	0.18 ± 0.10	0.200 ± 0.070	0.231 ± 0.046	0.281 ± 0.036	**0.398 ± 0.043**	0.451 ± 0.052	0.540 ± 0.025
	PRESTO	0.028 ± 0.030	0.064 ± 0.047	0.084 ± 0.077	0.099 ± 0.056	0.267 ± 0.058	0.323 ± 0.014	0.387 ± 0.038
	PRESTO-XTS	**0.095 ± 0.048**	**0.075 ± 0.044**	0.031 ± 0.033	0.013 ± 0.010	**0.28 ± 0.17**	**0.396 ± 0.055**	**0.510 ± 0.028**
	CrossPRESTO-XTS	0.071 ± 0.046	0.065 ± 0.087	**0.094 ± 0.072**	**0.119 ± 0.020**	0.255 ± 0.085	0.21 ± 0.16	0.498 ± 0.044

(c) Fine-tuning classification results for the **EuroCropsML S2** task solely on the `pasture meadow grassland grass` class.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.30 ± 0.32	**0.65 ± 0.26**	**0.850 ± 0.059**	0.871 ± 0.080	0.78 ± 0.01	0.701 ± 0.034	0.757 ± 0.036
	transfer learning (LV)	0.13 ± 0.22	0.21 ± 0.23	0.27 ± 0.19	0.44 ± 0.15	0.540 ± 0.088	0.582 ± 0.075	0.622 ± 0.047
	ANIL (LV)	0.28 ± 0.21	0.31 ± 0.28	0.68 ± 0.21	0.69 ± 0.19	0.79 ± 0.13	0.740 ± 0.030	0.766 ± 0.026
	MAML (LV)	0.21 ± 0.27	**0.47 ± 0.24**	0.529 ± 0.099	0.75 ± 0.12	**0.828 ± 0.062**	**0.858 ± 0.033**	**0.831 ± 0.069**
	FOMAML (LV)	0.35 ± 0.33	0.38 ± 0.20	**0.79 ± 0.12**	**0.81 ± 0.10**	0.72 ± 0.12	0.684 ± 0.045	0.764 ± 0.032
	TIML (encoder) (LV)	**0.47 ± 0.17**	0.39 ± 0.27	0.70 ± 0.01	0.72 ± 0.17	0.71 ± 0.13	0.766 ± 0.059	0.745 ± 0.052
	TIML (no encoder) (LV)	0.25 ± 0.33	0.32 ± 0.28	0.48 ± 0.23	0.67 ± 0.17	0.70 ± 0.14	0.661 ± 0.084	0.737 ± 0.038
	transfer learning (LV + PT)	0.007 ± 0.015	0.010 ± 0.011	0.30 ± 0.23	0.33 ± 0.19	0.489 ± 0.078	0.511 ± 0.064	0.686 ± 0.068
	ANIL (LV + PT)	0.14 ± 0.19	0.123 ± 0.073	0.43 ± 0.20	0.73 ± 0.12	0.77 ± 0.01	0.729 ± 0.086	0.756 ± 0.029
	MAML (LV + PT)	0.036 ± 0.050	0.133 ± 0.026	0.57 ± 0.21	0.66 ± 0.20	0.730 ± 0.087	0.704 ± 0.021	0.757 ± 0.025
	FOMAML (LV + PT)	0.043 ± 0.078	0.19 ± 0.17	0.54 ± 0.28	0.62 ± 0.17	0.703 ± 0.097	0.675 ± 0.069	0.744 ± 0.043
	TIML (encoder) (LV + PT)	0.15 ± 0.14	0.21 ± 0.14	0.60 ± 0.25	0.78 ± 0.11	0.75 ± 0.12	0.66 ± 0.15	0.786 ± 0.044
	TIML (no encoder) (LV + PT)	0.14 ± 0.26	0.176 ± 0.098	0.374 ± 0.060	0.62 ± 0.16	0.64 ± 0.15	0.632 ± 0.077	0.712 ± 0.040
	PRESTO	**0.30 ± 0.19**	0.42 ± 0.30	0.40 ± 0.13	0.57 ± 0.15	0.67 ± 0.14	0.67 ± 0.11	0.734 ± 0.043
	PRESTO-XTS	0.000 000	0.30 ± 0.35	**0.54 ± 0.22**	**0.911 ± 0.061**	0.75 ± 0.18	0.585 ± 0.057	**0.744 ± 0.019**
	CrossPRESTO-XTS	0.12 ± 0.26	**0.60 ± 0.47**	0.52 ± 0.23	0.886 ± 0.057	**0.757 ± 0.067**	**0.75 ± 0.21**	0.708 ± 0.061

is generally lower than in the **LV→EE** case (*cf.* Table 5a), with the exception of TIML (with encoder) in the 1-shot task. Furthermore, taking into account all classes (EE) as well as only overlapping classes (EE ∩ LV ∩ PT), all Portugal pre-training methods are even outperformed by the no-pretraining baseline. In both cases (**LV→EE** and **LV+PT→EE**), the few-shot performance of all algorithms is significantly lower when evaluated on those classes not previously seen during pre-training (EE \ LV and EE \ (LV ∪ PT)).

Table 5
Detailed analysis of supervised algorithms in the 20-shot **EuroCropsML S2** benchmark setting. The metric *classification accuracy* was evaluated on subsets of the test dataset corresponding to classes that are overlapping, overlapping excluding `pasture meadow grassland grass`, or non-overlapping (*cf.* Fig. 3) between the countries for the best performing pre-training scenario (in terms of accuracy on the validation dataset) per algorithm. We report the mean ± standard deviation over five repeated experimental runs in all cases on the test set. The best result per subset is marked in **bold**.

(a) **EuroCropsML S2** 20-shot **LV→EE** benchmark task.

	class subset	EE[a]	EE ∩ LV	(EE ∩ LV) \ {meadow}	EE \ LV
	algorithm				
accuracy	no pre-training	0.490 ± 0.020	0.542 ± 0.022	0.17 ± 0.12	0.021 ± 0.022
	transfer learning	0.365 ± 0.081	0.39 ± 0.10	0.33 ± 0.11	**0.15 ± 0.12**
	ANIL	0.490 ± 0.069	0.536 ± 0.077	**0.364 ± 0.064**	0.069 ± 0.014
	MAML	**0.507 ± 0.057**	**0.556 ± 0.063**	0.335 ± 0.058	0.055 ± 0.069
	FOMAML	0.476 ± 0.039	0.500 ± 0.090	0.205 ± 0.097	0.020 ± 0.022
	TIML (encoder)	0.423 ± 0.067	0.465 ± 0.078	0.178 ± 0.040	0.045 ± 0.037
	TIML (no encoder)	0.456 ± 0.058	0.499 ± 0.068	0.31 ± 0.10	0.062 ± 0.044

(b) **EuroCropsML S2** 20-shot **LV+PT→EE** benchmark task. Cases in which the **LV+PT→EE** was an improvement over the corresponding **LV→EE** result in Table 5 are highlighted with green color.

	class subset	EE[a]	EE ∩ LV ∩ PT	(EE ∩ LV ∩ PT) \ {meadow}	(EE ∩ LV) \ PT	(EE ∩ PT) \ LV	EE \ (LV ∪ PT)
	algorithm						
accuracy	no pre-training	**0.490 ± 0.020**	**0.648 ± 0.042**	0.079 ± 0.077	0.24 ± 0.15	0.016 ± 0.015	0.046 ± 0.058
	transfer learning	0.292 ± 0.089	0.28 ± 0.14	0.132 ± 0.028	0.415 ± 0.052	**0.095 ± 0.082**	0.170 ± 0.032
	ANIL	0.480 ± 0.045	0.563 ± 0.082	0.129 ± 0.060	0.413 ± 0.091	0.053 ± 0.060	0.19 ± 0.11
	MAML	0.452 ± 0.094	0.51 ± 0.14	**0.147 ± 0.044**	0.44 ± 0.10	0.035 ± 0.029	0.20 ± 0.11
	FOMAML	0.445 ± 0.058	0.49 ± 0.10	0.147 ± 0.078	0.475 ± 0.083	0.064 ± 0.093	0.17 ± 0.14
	TIML (encoder)	0.438 ± 0.031	0.582 ± 0.072	0.064 ± 0.048	0.20 ± 0.12	0.029 ± 0.037	0.036 ± 0.053
	TIML (no encoder)	0.441 ± 0.070	0.48 ± 0.11	0.119 ± 0.036	**0.494 ± 0.056**	0.047 ± 0.036	**0.221 ± 0.071**

[a] The results on all Estonia classes are identical with the results in Table 4.

6.1.2. Self-supervised learning

The SSL methods do not fully match the overall accuracy of the meta-learning methods, *cf.* Tables 4a to 4c. However, in terms of overall accuracy (Table 4a) and meadow-class accuracy (Table 4c), CrossPRESTO-XTS consistently outperforms standard transfer learning, except in two settings. Moreover, it shows the strongest SSL performance when it comes to overall accuracy, particularly in low-shot regimes ($2 \leq k \leq 20$).

When excluding the dominant class from the evaluation (Table 4b), all models drop in accuracy, yet SSL often outperforms the no-pretraining baseline on the minority classes. Notably, in the 500-shot regime, PRESTO-XTS and CrossPRESTO-XTS even exceed several meta-learning algorithms, with PRESTO-XTS achieving up to 13% higher accuracy.

On the contrary, while PRESTO-XTS shows strong performance on minority classes for 1- and 5-shots, it struggles to capture the `pasture meadow grassland grass` class for these scenarios, compared to its SSL counterparts, misclassifying all samples for the 1-shot task. Nevertheless, the algorithm's performance improves with an increasing number of samples per class, even showing the overall highest score for the 20-shot task.

6.2. Results on EuroCropsML S1+S2+ERA5

The baseline model trained on all data of the **EuroCropsML S1+S2+ERA5** task yields $a_{\text{OA}} = 0.768 \pm 0.010$, again serving as a general reference for evaluating the task's difficulty.

6.2.1. Self-supervised learning

The SSL methods struggle in low few-shot tasks when evaluated on overall accuracy, exhibiting a consistent deficit from the no-pretraining baseline (Table 6a). However, their performances improve significantly when exposed to a greater number of labeled samples. Consequently, both PRESTO and PRESTO-XTS ultimately surpass training from scratch.

Furthermore, an evaluation of minority-class performance (Table 6b) reveals that all SSL models exceed the baseline, often by a significant margin. It is noteworthy that CrossPRESTO-XTS demonstrates a strong advantage in low few-shot tasks ($k \leq 10$) as well as for 100 shots for minority classes, outperforming all other methods.

Concurrently, PRESTO-XTS demonstrates the highest overall and minority-class accuracy in the 500-shot task, with $a_{\text{OA}} = 0.615 \pm 0.026$ and $a_{\text{MCA}} = 0.540 \pm 0.016$, surpassing no-pretraining by 14% on minority classes. However, the model struggles significantly in low-shot regimes.

Notably, while all SSL are outperformed by the no-pretraining baseline for $1 \leq k \leq 20$ in the **EuroCropsML S1+S2+ERA5** scenario (Table 6a), this advantage is largely due to the baseline's strong bias toward the dominant meadow class (Table 6c).

7. Discussion

The results in Section 6, while showing that no single algorithm dominates across all scenarios, highlight that meta-learning approaches demonstrate distinct strengths in specific tasks.

7.1. Supervised learning

This section first analyzes the impact of pre-training on Latvian data and then examines how the inclusion of Portuguese data during pre-training affects fine-tuning performance.

7.1.1. Effect of pre-training on data from Latvia

Table 4a provides an overview of the benchmark results when transferring knowledge gained from Latvian data to Estonian data. As illustrated in Fig. 3, certain crop-type classes are shared between Latvia, Portugal, and Estonia, while others are not. It is therefore of interest to ascertain whether models demonstrate enhanced performance on classes they have already encountered during pre-training.

Table 5a compares performance on classes overlapping between Latvia and Estonia (EE ∩ LV) and those exclusive to Estonia (EE \

Table 6
Classification accuracy for the **EuroCropsML S1+S2+ERA5** benchmark. We report the classification accuracy on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per few-shot task is marked in **bold**.

(a) Fine-tuning classification results for the full **EuroCropsML S1+S2+ERA5** task.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	**0.22 ± 0.17**	**0.433 ± 0.062**	**0.29 ± 0.10**	**0.457 ± 0.021**	0.459 ± 0.036	0.493 ± 0.022	0.597 ± 0.034
	PRESTO	0.121 ± 0.046	0.229 ± 0.099	0.249 ± 0.028	0.361 ± 0.055	0.475 ± 0.044	**0.516 ± 0.031**	0.611 ± 0.015
	PRESTO-XTS	0.18 ± 0.19	0.398 ± 0.071	0.236 ± 0.053	0.434 ± 0.065	**0.479 ± 0.023**	0.497 ± 0.031	**0.615 ± 0.026**
	CrossPRESTO-XTS	0.13 ± 0.20	0.083 ± 0.013	0.146 ± 0.032	0.368 ± 0.069	0.433 ± 0.048	0.453 ± 0.061	0.595 ± 0.022

(b) Fine-tuning classification results for **EuroCropsML S1+S2+ERA5** task excluding the majority class `pasture meadow grassland grass`.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.031 ± 0.062	0.0025 ± 0.0053	0.030 ± 0.053	0.0056 ± 0.0068	0.14 ± 0.13	0.28 ± 0.18	0.472 ± 0.030
	PRESTO	0.025 ± 0.023	0.052 ± 0.045	0.087 ± 0.081	**0.124 ± 0.072**	**0.340 ± 0.088**	0.388 ± 0.059	0.462 ± 0.058
	PRESTO-XTS	0.0043 ± 0.0059	0.0012 ± 0.0011	0.043 ± 0.048	0.037 ± 0.049	0.27 ± 0.17	0.32 ± 0.21	**0.540 ± 0.016**
	CrossPRESTO-XTS	**0.059 ± 0.053**	**0.094 ± 0.047**	**0.119 ± 0.035**	0.087 ± 0.075	0.23 ± 0.15	**0.428 ± 0.088**	0.490 ± 0.018

(c) Fine-tuning classification results for the **EuroCropsML S1+S2+ERA5** task solely on the `pasture meadow grassland grass` class.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	No pretraining	**0.43 ± 0.39**	**0.90 ± 0.13**	**0.56 ± 0.27**	**0.950 ± 0.049**	**0.80 ± 0.16**	**0.73 ± 0.17**	0.733 ± 0.060
	PRESTO	0.23 ± 0.11	0.42 ± 0.24	0.43 ± 0.12	0.62 ± 0.19	0.62 ± 0.16	0.656 ± 0.075	**0.773 ± 0.076**
	PRESTO-XTS	0.38 ± 0.40	0.83 ± 0.15	0.45 ± 0.16	0.87 ± 0.18	0.71 ± 0.20	0.69 ± 0.21	0.696 ± 0.057
	CrossPRESTO-XTS	0.20 ± 0.45	0.071 ± 0.070	0.18 ± 0.10	0.67 ± 0.21	0.66 ± 0.25	0.480 ± 0.059	0.710 ± 0.035

LV). The findings suggest that, in general, meta-learning approaches, specifically those belonging to the MAML family, offer a slight advantage in both overall accuracy and performance on overlapping classes (EE ∩ LV), compared to both the no-pretraining baseline and standard transfer learning. This highlights the importance of domain similarity in few-shot learning. Most pre-training-based methods (with the exception of FOMAML) even exhibit better accuracy on classes exclusively present within Estonia (EE \ LV) in comparison to the no-pretraining baseline. This indicates that during pre-training, the model backbones learn transferable features useful for novel class adaptation. This effect is most pronounced for standard transfer learning, which achieves the highest accuracy (13.6 %, compared to the next best approach ANIL with 9.8 %) on the class `legumes harvested green`—the most abundant class in Estonia that is not present in Latvia.

The relative importance of the class distribution becomes clearer upon analysis of the classification accuracy on various subsets of classes (*cf.* Tables 4b, 4c, and 5a). Overall, the performance of all algorithms drops when the dominant class `pasture meadow grassland grass` is excluded, and increases when evaluated exclusively on this class. Nonetheless, transfer learning and ANIL appear to learn the most stable representation across all fine-tuning classes, particularly for those previously seen during pre-training (*cf.* Table 5a), rather than disproportionately relying on the majority class. Although both algorithms show increased accuracy on `pasture meadow grassland grass`, they also maintain the most stable performance on minority classes (Tables 4b and 5a) (EE ∩ LV and (EE ∩ LV) \ {meadow}), compared to overall accuracy. In particular, transfer learning achieves nearly equal performance on majority and minority class(es) in low-shot regimes. Only with an increasing number of samples per class does the model tend to focus more heavily on the dominant class.

The steepest performance decline upon removing the majority class is observed for TIML (with encoder) and the baseline model. This suggests that TIML's strong performance in the 1-shot task is largely due to its focus on the `pasture meadow grassland grass` class, which is further confirmed by its high accuracy for that class in Table 4c. Similarly, in the 5-, 10-, and 20-shot tasks, the no-pretraining baseline outperforms all supervised methods on the meadow class, implying it primarily defaults to predicting the majority class—explaining its strong overall accuracy in low-shot regimes.

7.1.2. Impact of adding Portugal pre-training data

Table 4a further evaluates the results of pre-training on data from both Latvia and Portugal, followed by fine-tuning on data from Estonia. Adding Portugal to the pre-training data reveals limitations in geographical knowledge transfer: in terms of overall accuracy, none of the algorithms successfully bridges the domain gap between Portuguese and Estonian data, likely due to geographic and spectral variations. There are several potential reasons why the addition of Portugal data may not lead to improved performance:

Climate differences Estonia and Latvia share similar climates, facilitating cross-region generalization. Portugal's Mediterranean climate introduces a greater feature distribution shift, potentially leading to model bias and reduced pre-training effectiveness.

Crop diversity As visualized in Fig. 3, Estonia and Latvia share more common crop types, while Portugal introduces classes absent in the Baltic states. This potentially leads to less effective knowledge transfer.

Agricultural practices Differences in crop management, such as irrigation and soil treatment, vary significantly between Portugal and the Baltic regions, making it difficult for models trained on one region to adapt to the other. Portuguese soils tend to be fairly arid, acidic, and rocky (Inácio et al., 2008) in contrast to the humid, sandy, or clay soils in Latvia (Karklins, 2002) and Estonia (Kmoch et al., 2021).

Growth cycle differences Climate-driven differences in crop phenology result in mismatched temporal patterns for otherwise similar crops, making it challenging for models to align temporal features between regions.

Nevertheless, Tables 4b and 4c reveal that adding Portuguese data reduces the tendency of models to overfit to the dominant class `pasture meadow grassland grass`. In fact, the performance on minority classes (Table 4b) often improves with the addition of Portugal data. However, these gains are insufficient to offset the broader

generalization gap introduced by the inclusion of geographically and phenologically dissimilar data.

Table 5b shows once more that when evaluating on Estonia-specific classes (EE \ (LV ∪ PT)), most meta-learning algorithms — except TIML with encoder — and conventional transfer learning outperform the baseline by a significant margin. While a performance drop remains (as also seen in Table 5a for EE \ LV), it is less severe. Along with the improved performance on minority classes, this suggests that in the **LV+PT→EE** setting, the models shift their focus toward classes unique to Estonia. A likely explanation is that, while adding Portuguese data increases the total number of shared classes between pre-training and fine-tuning, the spectral characteristics of classes shared between Portugal and Estonia can differ substantially. This domain mismatch may introduce ambiguity, encouraging the models to focus more on domain-consistent classes — *i.e.*, those specific to Estonia. At the same time, exposure to more diverse pre-training data may help the models to learn broader, more generalizable features, mitigating overfitting to the ubiquitous dominant class `pasture meadow grassland grass` — shared across all countries — improving their ability to adapt to previously unseen Estonian classes.

7.1.3. Limitations of the TIML family

The algorithms of the TIML family, despite having incorporated additional parcel location information, show rather mediocre performance on our multi-class cross-regional benchmark. Although the original implementation (Tseng et al., 2022) successfully applied TIML on a binary classification task, our extension of applying it on a multi-class benchmark reveals significant challenges.

This might be attributed to several compounding factors. We are going to shortly revisit how location information is incorporated into the TIML algorithm:

TIML (no encoder) The static location information is simply appended to the time series as additional channels and passed directly to the model.

TIML (encoder) The static location information is treated as direct input features, which are simply passed through a standard series of linear blocks. This encoded information is then used to generate the γ (gain) and β (bias) parameters for the model backbone. As a result, the model's behavior is directly and solely conditioned on the precise location data of the current task.

First, cross-country transfer already introduces a substantial domain shift, which seems to be further amplified by adding precise parcel location features. The models tend to overfit to region-specific patterns that do not generalize well—especially when incorporating Portuguese parcel locations, which may encode localized agricultural characteristics that are not representative of Estonian conditions. This issue is particularly pronounced in the TIML encoder, where the model generates two distinct sets of γ and β for two locations that have slightly different coordinates. Hence, the encoder's architecture likely prioritizes capturing fine-grained, region-specific features at the expense of spatial generalization. As a result, the encoder may overfit to the pre-training regions and struggle to adapt to new geographical domains during fine-tuning. This raises the question to what extent the encoder can effectively leverage the knowledge acquired from the pre-training on distant location information during fine-tuning. Likewise, adding static location data as features to temporally structured time-series data, as done in TIML (without encoder), can cause the model to over-rely on the coordinates as a unique identifier, thereby obscuring the inherent temporal patterns necessary for crop phenology analysis. In summary, in both TIML setups, the model is highly sensitive to the exact spatial coordinates rather than generalizing to broader agro-ecological or climatic regions.

Moreover, TIML—in particular with encoder—appears to struggle slightly more with the dataset's class imbalance, compared to transfer and other meta-learning variants. A likely explanation is that the dominant meadow class is geographically ubiquitous in Estonia. Hence, from the model's perspective, many different locations correlate with the same label, making location features a poor discriminator. Rather than aiding class separation, they become a reinforcing signal to default to the majority class. This suggests that the algorithm does not explicitly learn features that help distinguish minority classes, instead emphasizing broadly applicable, but ultimately uninformative, patterns.

In summary, the combination of these factors — cross-regional domain shifts, over-reliance on geographically anchored features, and encoder architectural limitations — hinders TIML's effectiveness in transferring knowledge across such heterogeneous datasets. These findings also raise broader concerns about the utility of geographical location features in classifying different crop types, given that similar latitudes can show divergent agricultural practices. However, a more detailed study of the TIML encoder algorithm is beyond the scope of this work and will be left for future research.

7.1.4. Run-time analysis

Despite their superior performance, meta-learning algorithms tend to require longer training periods and greater computational resources compared to baselines. While ANIL and FOMAML significantly reduce computational costs compared to the standard MAML algorithm, run time remains one of their major drawbacks.

Run times of the meta-learning algorithms are fairly stable across the two tasks with and without incorporating data from Portugal. In contrast, the run times of transfer learning increase substantially when incorporating the additional data from Portugal. A detailed run-time analysis can be found in Table C.24 in Appendix C.

7.2. Self-supervised learning

The results presented in Section 6 indicate that, while self-supervised learning does not consistently match the overall accuracy of meta-learning approaches, they nonetheless offer meaningful advantages over certain methods, particularly in low-shot and minority-class scenarios. Moreover, although the no-pretraining baseline regularly outperforms SSL in the **EuroCropsML S1+S2+ERA5** task (*cf.* Table 6a), it achieves its high overall accuracy by disproportionately focusing on the majority class, *cf.* Table 6c. Conversely, Table 6b clearly demonstrates that the SSL models achieve significantly better performance on the remaining classes. This is also confirmed by the superior performance on Cohen's kappa, *cf.* Table B.23 in Appendix B.5. Therefore, while the baseline initially appears to perform better, it fails to predict the individual classes. We conclude that for the crucial task of individual crop classification, the advanced SSL models are indeed performing better.

Moreover, all PRESTO variants seem to be fairly robust to missing input data. The robustness to incomplete spectral information is most evident in the **EuroCropsML S2** task, where fewer spectral bands are available than during pre-training. Despite the reduced spectral information, the SSL models maintain competitive performance, suggesting that their learned representations are resilient to input degradation. Despite being significantly computationally more efficient during pre-training (*cf.* Appendix C), CrossPRESTO-XTS also frequently outperforms PRESTO-XTS in the **EuroCropsML S2** task. Furthermore, it consistently (with only one exception) achieves higher minority-class accuracy in the **EuroCropsML S1+S2+ERA5** scenario (Table 6b), indicating improved generalization to underrepresented classes.

7.3. Comparison of supervised and self-supervised methods

While SSL on general-purpose land cover data enables moderate generalization, it generally lags behind meta-learning algorithms pre-trained on crop-specific data—especially in lower-shot regimes ($k \leq 100$). This reflects the importance of task-specific inductive biases and domain relevance in few-shot settings.

Supervised and, in particular, meta-learning algorithms are explicitly designed to adapt rapidly to new tasks using minimal labeled data, making them particularly well-suited for crop-type classification problems, which often involve new regions, growing seasons, or crop varieties. In contrast, SSL focuses on extracting broad, task-agnostic features from unlabeled data, which, while powerful, may lack the semantic specificity required for effective domain adaptation in agricultural settings with limited samples.

However, SSL often outperforms the no-pretraining baseline, with the greatest improvements observed for underrepresented classes as well as higher-shot settings. It also tends to surpass traditional transfer learning methods in overall performance.

8. Conclusion

This study presented a comprehensive benchmark on the newly introduced EuroCropsML dataset, demonstrating its use for benchmarking few-shot crop-type classification by evaluating the effectiveness of both supervised and self-supervised methods. It sets a precedent for future benchmarks in agricultural remote sensing, providing a valuable resource to evaluate novel machine learning algorithms or models on real-world, multi-class, and multi-region crop type data. The evaluation spans diverse European regions, offering new insights into the effectiveness of these methods for real-world remote sensing applications in agriculture.

The advances detailed in this work directly enable a new era of realistic crop-type classification for end users across Europe. By accurately reflecting real-world conditions in our benchmark, we successfully bridge the critical gap between technical research and practical applications. This is vital because high-performing models, proven to be robust in real-world settings, enable farmers and decision-makers to draw meaningful, actionable conclusions, leading to the immediate adoption of precision farming techniques and consequential cost reductions.

The results indicate that while meta-learning algorithms from the MAML family generally outperform transfer learning in terms of accuracy, they come with increased computational costs and training time. The TIML family of algorithms did not show a significant improvement in prediction accuracy. Incorporating additional data from a more distant geographical country into the pre-training stage did not lead to enhanced performance. This indicates that regional disparities may not be conducive to effective knowledge transfer and serves to illustrate the inherent difficulties associated with transferring knowledge between disparate geographical regions.

In general, the study demonstrates the complexity of applying meta-learning techniques to geospatial problems. Although meta-learning can adapt to variations within similar regions, transferring knowledge between regions with substantial differences requires careful consideration of the characteristics of the underlying data. These findings highlight the trade-offs between computational efficiency and prediction accuracy in machine learning applications for remote sensing and crop type classification. Hence, we encourage researchers to first investigate regular transfer learning approaches to establish a baseline for cross-country adaptation before exploring whether meta-learning techniques could improve classification performance.

Furthermore, crop classification often requires distinguishing between visually and spectrally similar classes, such as `wheat` and `barley`. Prior studies (Stiller et al., 2024; Marszalek et al., 2022) show that SSL models tend to underperform on such fine-grained distinctions, as they are not trained to optimize for class-level separation. Similar trends have been observed in, for instance, plant phenotyping tasks, where SSL models were found to be more sensitive to redundancy in pre-training datasets (Ogidi et al., 2023). Thus, while SSL is effective at leveraging unlabeled data, supervised methods currently provide a more robust solution for crop-type classification under real-world constraints.

Nevertheless, in the absence of labeled pre-training data, SSL remains a valuable strategy. Our experiments show that, especially for minority classes and with an increasing number of training samples, SSL often yields substantial improvement over training from scratch. In addition, it frequently outperforms conventional transfer learning.

Finally, the limitations of pre-training in general are compounded by this study's real-world evaluation design. While low-shot training splits are few-shot and mostly class-balanced, validation and test sets retain their original, highly imbalanced distributions—reflecting realistic deployment scenarios. This poses a substantial challenge for both supervised and self-supervised methods to maintain performance on minority classes.

8.1. Future work, limitations, and recommendations

Our comprehensive evaluation naturally reveals certain trade-offs and constraints inherent to working with real-world satellite data and advanced ML methods. In this section, we first detail the limitations of the EuroCropsML dataset before offering specific recommendations and directions for methodological future work.

8.1.1. EuroCropsML

While EuroCropsML is a highly valuable resource for benchmarking real-world crop-type classification, we would like to point out two limitations. It does not currently provide multi-year data, preventing a model performance analysis across multiple years. However, both EuroCrops and EuroCropsML datasets are part of ongoing projects and are still continuously developed, so that extensions of the data to cover additional years can be expected in the future. Furthermore, the spatial coverage of EuroCropsML includes only three countries of the EU. Although these countries were carefully selected based on specific factors to reflect real-world scenarios (Reuss et al., 2025b), they remain spatially constrained and do not encompass all agricultural and climatic zones in Europe.

The primary goal of our study is to determine whether specific algorithms perform well in challenging real-world scenarios. These include cross-regional transfer, which inherently involves geographic variations in phenology, cultivation practices, and differing climates, as well as handling highly imbalanced and heterogeneous crop types. If a method proves robust here, it serves as a strong foundation for a full-scale EuroCrops investigation.

8.1.2. Methodology

As mentioned in Section 7.1.3, we generally do not recommend incorporating location information as done in TIML, especially when dealing with spatially heterogeneously distributed classes. Nevertheless, we acknowledge the idea of TIML to add task information to the meta-learning task. However, we suggest exploring the replacement of direct spatial coordinate encodings with trainable geographic embeddings or semantic environmental features, such as temperature or altitude, to significantly improve the adaptability of TIML-like models.

In general, while meta-learning can lead to improved performance, the training is accompanied by extremely high computational costs. Therefore, future work should focus on exploring and implementing more memory-efficient meta-learning algorithms and optimization techniques to fully leverage their potential.

The inherent high class imbalance in EuroCropsML is characteristic of real-world crop-type datasets and poses a considerable challenge to all investigated methods. Moreover, particularly in low-shot regimes, the methods struggle with learning robust representations. Therefore,

there is a significant need to explore further techniques, such as class-balanced sampling, few-shot adaptation, or reweighting strategies, to bias the learning process toward less common classes, as well as to create more discriminative and robust representations, *cf.* Reuss et al. (2025a).

CRediT authorship contribution statement

Joana Reuss: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jan Macdonald:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Simon Becker:** Writing – review & editing, Writing – original draft, Visualization, Validation, Investigation, Formal analysis, Conceptualization. **Ekaterina Gikalo:** Writing – original draft, Visualization, Software, Methodology, Data curation. **Konrad Schultka:** Software, Methodology, Conceptualization. **Lorenz Richter:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Marco Körner:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization.

Acknowledgments

The project is funded by the German Federal Ministry for Economic Affairs and Energy based on a decision by the German Bundestag under the funding reference 50EE2007B (J. Reuss, E. Gikalo, and M. Körner), 50EE2007A (J. Macdonald, S. Becker, K. Schultka, and L. Richter) and 50EE2105 (M. Körner). S. Becker acknowledges support by the Swiss National Science Foundation Grant PZ00P2 216019.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Implementation details

We implement all algorithms within the `PyTorch` framework (Paszke et al., 2019) for machine learning and automatic differentiation.

A.1. Neural network architectures

For all experiments, we use a state-of-the-art Transformer architecture. During supervised pre-training, we use a single Transformer encoder with a linear classification layer on top, which predicts the class log-probabilities. When pre-training in an SSL manner, we use a full Transformer encoder–decoder architecture. Specifically, we use a single Transformer block for both the encoder and decoder, comprising four attention heads. Each token in the input sequence is represented by an internal embedding vector of dimension 128. The fully connected network within a Transformer block uses a hidden dimension of $d_{\text{hidden}} = 256$, expanding the initial embedding dimension by a factor of two. Additive sinusoidal temporal positional encoding (Vaswani et al., 2017) with a maximum sequence length of 366, accommodating daily samples over a full year of temporal coverage, including leap years, is used.[2]

[2] Even though we are only looking at data from 2021, setting the maximum length of the positional encoding to 366 (leap year) provides a more flexible approach for further use cases. For instance, it allows fine-tuning of models that were pre-trained on leap year data

A.1.1. PRESTO

The original PRESTO implementation uses a Transformer encoder–decoder architecture with an embedding dimension of $d_{\text{emb}} = 128$, defining the size of feature vectors throughout the Transformer blocks. The model comprises four Transformer blocks in total — two in the encoder and two in the decoder — each using eight attention heads. The fully connected network within each block uses a hidden dimension of $d_{\text{hidden}} = 512$, expanding the initial embedding dimension by a factor of four. Additive sinusoidal temporal positional encoding (Vaswani et al., 2017) with a maximum sequence length of 24 is used, corresponding to a monthly sampling over two consecutive years.

A.2. Model training

The general training follows the two-phase approach of pre-training and fine-tuning. Besides the actual learnable parameters of the Transformer model (weights), there are a number of non-learned hyperparameters that influence the model. These include hyperparameters that were heuristically/empirically selected (batch size for the mini-batch gradient descent, annealing schedules for learning rates), as well as hyperparameters that were automatically tuned over numerous trial runs (learning rates). The term "empirically selected" denotes that experiments were conducted with a range of values for the respective hyperparameter, with the optimal value being determined on the basis of the validation accuracy. Our primary objective was to minimize the total runtime by significantly reducing the number of tuning trials required. Consequently, we fixed all hyperparameters except for the learning rates, which were the only parameters subjected to tuning. The heuristically set values, along with the valid ranges for tuned and empirically determined hyperparameters, were established on the basis of initial experiments, which showed the greatest improvements with these settings (*cf.* Tables A.7, A.12 and A.14). Subsequently, the heuristically set hyperparameters remained constant throughout the experimental process. For each of the empirically configured hyperparameters, we executed the full set of experiments — including the tuning of learning rates — which yielded multiple trained networks. The following subsections each contain an overview of the (tuned/selected) hyperparameters. For the tuning trial runs, we employ a *Bayesian Tree-Parzen Estimator (TPE)* with percentile pruning within the `Optuna` framework (Akiba et al., 2019). We employ a total of 100 warm-up steps and keep the top 75th percentile of runs. In all cases, the selection of the best hyperparameters is based on the classification accuracy on the validation dataset. This is also used to determine the best pre-training epoch. Its model state is used to initialize the model backbone weights before fine-tuning. Finally, the best fine-tuning epoch is determined by the classification accuracy on the validation dataset of the fine-tuning task. Its model state is used to compute the final reported model accuracy on the fine-tuning test dataset.

A.3. Meta-learning

All meta-learning algorithms are trained using a mini-batch stochastic gradient descent in the inner optimization loop and the Adam optimizer (Kingma and Ba, 2017) in the outer optimization loop. The learning rates for the inner and outer optimization loops are individually tuned on 20 000 meta-train tasks. We then meta-train the models on 100 000 meta-training tasks and validate after every 100 tasks on the meta-validation tasks (corresponding to epochs for regular training). The classification head layer is adapted to predict a vector in $\mathbb{R}^{n_c}$ of n_c class logits that correspond to the distinct classes. The weights of the classification head layer are reset at the start of each new inner optimization loop to allow it to adapt to each new batch of tasks. As mentioned in Section 5, we carry out a series of experiments with different n-way k-shot settings and inner loop steps s, *cf.* Table A.7. Subsequently, all meta-trained models undergo fine-tuning on the Estonia data.

Table A.7
Meta-learning hyperparameters. The hyperparameters employed during meta-learning, along with their corresponding valid values, were either set heuristically/empirically or were automatically tuned.

Hyperparameter	Value/value range	Assignment
tuning tasks	100 000	heuristical
tuning trials per setup	36	heuristical
inner learning rate α	$\alpha \in [0.01, 10.0]$	tuned
outer learning rate β	$\beta \in [0.0001, 0.1]$	tuned
encoder learning rate η[a]	$\eta \in [0.0001, 0.1]$	tuned
training tasks	100 000	heuristical
number of tasks per batch	4	heuristical
number of classes per task n	$n \in \{4, 10\}$	empirical
number of samples per class k	$k \in \{1, 10\}$	empirical
number of inner loop gradient steps s	$s \in \{1, 4, 10\}$ for $n = 4$, $s = 1$ for $n = 10$	empirical

[a] The encoder learning rate corresponds to the learning rate of the TIML encoder and, hence, was solely employed for the purpose of TIML (with encoder) training.

A.3.1. Suggestions on meta-learning hyperparameters

As mentioned in Appendix A.2, the hyperparameters were either set heuristically, empirically, or automatically tuned. Tables A.8–A.11 show the final selected learning rates during meta-learning. As has been found in prior research (Triantafillou et al., 2020), the inner learning rate, denoted by α, benefits from being set to relatively high values. This is because the inner loop performs task-specific adaptations on a small amount of data for a limited number of steps. Hence, it needs to take larger, more aggressive steps to quickly shift the model parameters from their meta-initialization point to the task-adapted point. Notably, in most cases, an increase in the number of samples k or the inner steps taken s with a constant number of classes n will result in a decrease in the inner learning rate. Conversely, increasing the number of classes n (while keeping k and s constant) will result in a significant increase for α.

The outer loop, on the other hand, performs meta-learning optimization based on the loss calculated after adaptation. This is an averaging process across many tasks. Therefore, the outer learning rate, denoted by β, exhibits a greater similarity to conventional learning rates and should be assigned a relatively low value to slowly converge to a good, versatile starting point across the entire task distribution.

Similarly, the TIML encoder learning rate (*cf.* Tables A.9 and A.11) also requires relatively low values. This is because the encoder's role is to learn general knowledge that is stable across tasks, making it part of the meta-knowledge update during the outer loop optimization.

A performance evaluation of the various n-way k-shot s-step settings can be found in Appendix B.3. The values for n, k, and s were set based on previous work, *cf.* Finn et al. (2017), Triantafillou et al. (2020) and Rußwurm et al. (2020b).

It should be noted that, while the provided values are applicable to our benchmark, they may not be directly transferable to other use cases. Consequently, we strongly encourage future researchers to undertake their own experimental investigations.

A.4. Transfer learning

The regular transfer learning experiments are conducted using the Adam optimizer (Kingma and Ba, 2017) with a tuned learning rate and a batch size of 128. We train for a maximum of 150 epochs on the training set, validate after each epoch on the validation set, and stop the training in case the validation loss does not decrease for more than 15 epochs (early stopping). Hyperparameter tuning and training are performed with varying batch sizes $b \in \{4, 8, 16, 32, 64, 128, 264\}$. Based on the validation accuracy, the batch size 128 is selected for further experimentation and subsequent fine-tuning. Hence, with a batch size of 128, we performed a series of experiments with the cosine annealing learning rate schedule. The number of cycles is set to zero (corresponding to no cosine annealing), one, or two cycles. We choose the most promising schedule based on the validation accuracy, which is one annealing cycle when pre-training only on data from Latvia, and no annealing when pre-training on data from Latvia and Portugal. Table A.12 contains an overview of all hyperparameters used during transfer learning.

Table A.8
Meta-learning learning rates for LV. All listed runs were used at least once for the final results in Section 6.

Algorithm	n	k	s	α	β
ANIL	4	1	1	0.4406	0.0006
	4	1	4	0.2610	0.0007
	4	1	10	0.3136	0.0008
	4	10	1	0.5031	0.0012
	10	1	1	2.7327	0.0014
MAML	4	1	1	0.1706	0.0005
	4	1	4	0.0879	0.0008
	4	1	10	0.0108	0.0012
	10	1	1	0.3582	0.0014
	10	10	1	0.0612	0.0008
FOMAML	4	1	1	0.1068	0.0024
	4	1	4	0.0254	0.0002
	4	1	10	0.0294	0.0003
	4	10	1	0.0351	0.0004
	10	1	1	0.1855	0.0027
	10	10	1	0.3670	0.0012

Table A.9
TIML meta-learning learning rates for LV. All listed runs were used at least once for the final results in Section 6.

Encoder	n	k	s	α	β	η
yes	4	1	1	0.5782	0.0006	0.0004
	4	1	10	0.0409	0.0001	0.0007
	4	10	1	0.0759	0.0002	0.0010
	10	1	1	0.4189	0.0004	0.0004
	10	10	1	0.1212	0.0008	0.0029
no	4	1	1	0.6658	0.0008	–
	4	1	4	0.0412	0.0005	–
	4	1	10	0.0238	0.0007	–
	4	10	1	0.1141	0.0003	–
	10	1	1	0.5062	0.0015	–

Table A.10
Meta-learning learning rates for LV+ PT. All listed runs were used at least once for the final results in Section 6.

Algorithm	n	k	s	α	β
ANIL	4	1	1	0.2178	0.0014
	4	1	4	0.2164	0.0010
	4	1	10	0.0982	0.0030
	4	10	1	0.5914	0.0001
	10	1	1	2.8254	0.0029
	10	10	1	0.9356	0.0009
MAML	4	1	1	0.4699	0.0001
	4	1	4	0.0191	0.0003
	4	1	10	0.0233	0.0008
	4	10	1	0.8820	0.0121
	10	1	1	0.5760	0.0016
	10	10	1	0.3433	0.0008
FOMAML	4	1	1	0.4699	0.0001
	4	1	4	0.0294	0.0003
	4	1	10	0.0351	0.0004
	4	10	1	0.1976	0.0007
	10	1	1	0.6839	0.0007
	10	10	1	0.9523	0.0008

A.5. Self-supervised learning

In alignment with the aforementioned transfer learning, all SSL models are pre-trained using the Adam optimizer with a tuned learning

Table A.11
TIML meta-learning learning rates for LV+ PT. All listed runs were used at least once for the final results in Section 6.

Encoder	n	k	s	α	β	η
yes	4	1	1	0.2178	0.0006	0.0001
	4	10	1	0.2873	0.0006	0.0017
	10	1	1	1.4578	0.0003	0.0005
	10	10	1	0.2534	0.0004	0.0021
no	4	1	4	0.1000	0.0001	–
	4	1	10	0.0282	0.0005	–
	10	1	1	0.4474	0.0012	–

Table A.12
Transfer learning hyperparameters. The hyperparameters utilized during transfer learning, along with their corresponding valid values. They were either set heuristically or empirically, or alternatively, were automatically tuned.

Hyperparameter	Value/Value range	Assignment
tuning epochs	150	heuristical
tuning trials per setup	36	heuristical
training epochs	150	heuristical
early stopping patience p (epochs)	15	heuristical
batch size b	$b \in \{1, 4, 16, 32, 64, 128, 264\}$	empirical
learning rate α	$\alpha \in [0.0001, 0.01]$	tuned
cosine annealing cycles c	$c \in \{0, 1, 2\}$	empirical

Table A.13
Self-supervised learning hyperparameters. The hyperparameters utilized during SSL pre-traning, along with their corresponding valid values. They were either set heuristically or empirically, or alternatively, were automatically tuned.

Hyperparameter	Value/Value range	Assignment
tuning trials per setup	8	heuristical
early stopping patience p (steps)	15	heuristical
batch size b	$b \in \{128, 256\}$	empirical
learning rate α	$\alpha \in [0.00001, 0.01]$	tuned

rate (*cf.* Appendix A.2) and a batch size of 256 for one epoch. After 250 000 training batches (steps), we validate on the validation data. Hyperparameter tuning and training are performed with varying batch sizes $b \in \{128, 256\}$. The selection of batch sizes and number of epochs is based on the computational constraints provided by the system. Based on the validation accuracy, the batch size 256 is selected for further experimentation (*cf.* Appendix B) and subsequent fine-tuning. Table A.13 contains an overview of all hyperparameters used during self-supervised learning.

A.6. Fine-tuning

For the models that were pre-trained via SSL, we replace the models' decoder with a simple linear classification layer to predict class log-probabilities. For the meta-learned and pre-trained models, we simply reset the classification head layer. All models are fine-tuned for a maximum of 200 epochs on Estonia tasks using an Adam optimizer (Kingma and Ba, 2017). We validate every epoch on the validation set. Training is stopped if the validation loss does not decrease for more than five epochs (early stopping). Furthermore, we adjust the learning rate(s) in three different settings as described in Section 5.2. Tables B.18 and B.19 illustrate the results of the 20-shot task, exemplifying the impact of the above learning rate settings.

All experiments are repeated five times with five different random seeds $r \in \{0, 1, 42, 123, 1234\}$. We set the random seed across multiple libraries, including Python's built-in `random` module, `NumPy`'s random number generator, and `PyTorch`'s CPU random number generator. Moreover, we mandate that `PyTorch` utilizes deterministic algorithms for operations on the GPU when using CuDNN, and disable the auto-tuning functionality of CuDNN. Hence, for each random seed r, the experiments are almost entirely deterministic. The only remaining randomness is introduced by `Optuna`'s TPE during hyperparameter tuning. An overview of the relevant hyperparameters is shown in Table A.14.

Appendix B. Additional results

The results in Tables 4 and 6 show the best-performing setup for each model and algorithm, selected based on overall accuracy a_{OA} on the validation set. This section provides a more comprehensive overview and evaluation. In particular, we

- evaluate different strategies for hyperparameter tuning,
- analyze model performance on broader parent-level classes,
- assess the impact of meta-learning parameters in an n-way k-shot s-step setting,
- investigate the impact of specific masking strategies during self-supervised pre-training,
- assess fine-tuning performance using a larger Transformer encoder–decoder architecture for SSL,
- and analyze robustness under class imbalance using additional evaluation metrics.

B.1. Impact of learning rate tuning

During fine-tuning, we considered three different hyperparameter tuning scenarios for the learning rate(s) of each algorithm, *cf.* Section 5.2: using the *same* learning rate for the *head* & *backbone*, allowing *different* learning rates for the *head* & *backbone*, and *only* tuning a learning rate for the *head* while keeping the backbone fixed. Tables B.18 and B.19 illustrate the results for each algorithm on the 20-shot fine-tuning benchmark tasks for the three learning rate settings.

B.1.1. Supervised learning

Fine-tuning both the model backbone and the model head achieved higher prediction accuracies compared to only fine-tuning the model head in all cases. In addition, the most flexible setting with separately tuned learning rates for the model backbone and head performed best in the majority of cases. Noteworthy, the ANIL algorithm, which only adapts the learnable parameters of the model head during its inner optimization loop, did not perform better than the other transfer meta-learning approaches when fine-tuning only the model head. While in theory using just one learning rate is less flexible than allowing separate ones for head and backbone, using just one might still turn out to be advantageous: If the backbone and head use different learning rates, the gradient updates may vary significantly between the two, leading to an imbalance in how quickly each part adapts during training. This can cause the model to optimize one part faster than the other, potentially leading to suboptimal representations in the backbone or overfitting in the head.

B.1.2. Self-supervised learning

All methods, except PRESTO-XTS for the **EuroCropsML S2** task, achieved their highest accuracy by training both head and backbone, either with the same or with separate learning rates. Moreover, the majority of algorithms demonstrated enhanced performance when equipped with distinct learning rates for backbone and head. On the **EuroCropsML S1+S2+ERA5** task in particular, all SSL methods performed substantially better with separate learning rates. At the same time, the no pre-training baseline achieved its best performance with a single, unified learning rate for both head and backbone. Overall, this behavior is expected: the backbone of the pre-trained models already contains learned representations, while the classification head starts from a random initialization for all models, and hence benefits from a separate, potentially higher, learning rate.

Table A.14
Fine-tuning hyperparameters. The hyperparameters utilized during fine-tuning, along with their corresponding valid values. Their final value was either assigned heuristically or empirically, or alternatively, was automatically tuned. The optimal batch size for fine-tuning, denoted by b, was identified through a process of fine-tuning the transfer learning models with varying fine-tuning batch sizes. Subsequently, the final batch size of 16 was selected based on the validation accuracy observed during the fine-tuning process.

lr tuning Hyperparameter	only head value/value range	different head & backbone value/value range	same head & backbone value/value range	Assignment
tuning epochs	200	200	200	heuristical
tuning trials per setup	8	32	8	heuristical
training epochs	200	200	200	heuristical
early stopping patience k (epochs)	5	5	5	heuristical
batch size b	16	16	16	empirical
learning rate α	$\alpha_{\text{head}} \in [10^{-6}, 10^{-2}]$, $\alpha_{\text{backbone}} = 0$	$\alpha_{\text{head}} \in [10^{-6}, 10^{-2}]$, $\alpha_{\text{backbone}} \in [10^{-6}, 10^{-3}]$	$\alpha \in [10^{-6}, 10^{-2}]$	tuned

B.2. Performance on broader parent-level classes

The hierarchical class structure of EuroCrops supports performance analysis at multiple class levels. This section outlines how the EuroCropsML dataset can be used for more detailed evaluation and error analyses. For the main evaluation in Section 6, we used the final HCAT3 level 6. Similarly, Tables B.15 and B.16 present classification accuracy based on HCAT3 levels 3 and 4. In general, a prediction is considered correct if the predicted parent class at level d matches the ground truth's parent class at the same level. Level 3 includes six parent classes — specifically `mushrooms energy genetically modified crops`, `tree wood forest`, `arable crops`, `pasture meadow grassland grass`, `permanent crops perennial`, `not known and other` — while level 4 comprises 33. It is worth noting that the `pasture meadow grassland grass` class is a standalone category without any parent or child classes. As such, it is present in level 3, level 4, and the final fine-grained class set used in Section 6. Interestingly, despite the algorithms of the MAML family trained on LV still showing superior performance for both parent levels 3 and 4, SSL methods often outperform supervised methods trained on LV+ PT, particularly in low-shot regimes ($k \leq 5$). This is likely due to the Portugal samples distorting the representation learned during pre-training. Hence, the model requires more data to accurately adjust the weights during fine-tuning. Conversely, the results indicate that SSL successfully learns robust, general representations that inherently benefit broader, parent-level classes CrossPRESTO-XTS, in particular, again achieves acceptable performance across all few-shot settings. While we do not discuss these results in more detail here, they illustrate one way the dataset can support multi-level performance analysis.

B.3. Evaluation of meta-learning hyperparameters

Table B.17 shows the classification accuracy of the 20-shot benchmarking tasks for the six n-way k-shot s-step settings of the meta-learning algorithms. Although there was no single setting that performed best in all algorithms and benchmark tasks, in most cases, the best prediction accuracies could be achieved for meta-learning in 4-way 1-shot 4-step. This shows that all meta-learning algorithms could take advantage of taking more than one gradient step during the inner optimization for the task adaptation, but a small number of steps is sufficient. Furthermore, allowing for a larger number of classes ($n = 10$) and thus potentially a higher diversity between classes in tasks during pre-training did not improve downstream fine-tuning performance for any algorithm. Similarly, presenting the models with more samples per class ($k = 10$) and thus potentially a higher intra-class diversity in the tasks during pre-training also did not improve their fine-tuning performance in all but one case.

B.4. Effect of masking design and model capacity on PRESTO

In addition to using a mixture of all masking strategies defined in Section 4.3.3, we also experimented with restricting the model to deterministic masking only, *i.e.*, masking entire channel groups and contiguous time steps. This design choice is motivated by real-world remote sensing deployment scenarios, where entire input modalities may be missing at test time, *e.g.*, due to persistent cloud cover in Sentinel-2, and temporal gaps often occur in contiguous blocks. By pre-training the model under these deterministic masking conditions, we aim to encourage the encoder to learn more robust representations.

Moreover, along with the model defined in Appendix A.1, we pre-train a larger variant of PRESTO-XTS and CrossPRESTO-XTS, with a size that matches the original PRESTO model (*cf.* Appendix A.1.1) but uses an increased embedding size of $d_{\text{emb}} = 256$. This choice was made based on the fact that bigger models have the capacity to often learn richer representations during self-supervised pre-training. Tables B.20 and B.21 show the results for both the main models and the bigger variant. Models using the larger architecture are marked with the postfix *L*, and those using deterministic masking with the postfix *D*.

Across both fine-tuning tasks, increasing the model size and using deterministic masking strategies significantly influence performance. For **EuroCropsML S2** low-shot regimes (Table B.20a), PRESTO-XTS (D, L) and CrossPRESTO-XTS (D, L) generally improve overall accuracy compared to Table 4. In particular, the increased model size seems to boost performance here. However, especially CrossPRESTO-XTS tends to focus mostly on the majority class when trained with a larger model, as reflected in Table B.20b. As the number of training samples increases, the original PRESTO-XTS model frequently outperforms the larger or deterministic variants.

For the multi-modal **EuroCropsML S1+S2+ERA5** task, PRESTO-XTS consistently favors the original model (Section 6) from five shots onward, while its minority-class accuracy improves most with the smaller model for higher shot regimes. In contrast, CrossPRESTO-XTS retains a longer advantage from the larger model size for overall accuracy, suggesting its pre-training architecture scales better with multi-modal complexity. However, for minority-class performance, the smaller CrossPRESTO-XTS variant again performs best, which is consistent with trends observed in **EuroCropsML S2**.

All in all, both strategies appear to help increase accuracy in low-shot regimes but often increase the model's tendency of focusing on the majority class instead of improving per-class predictions.

B.5. Robustness evaluation under class imbalance

While the main evaluation in Section 6 focuses on the overall classification accuracy and a restricted accuracy excluding the majority class that addresses the high class imbalance in the data, we report *Cohen's kappa coefficient* κ as an additional evaluation metric. The kappa score offers a complementary perspective to accuracy by accounting for chance agreements between predicted and ground truth labels, providing a robust metric in scenarios with skewed class distributions. Divergence between a_{MCA} (*cf.* Tables 4b and 6b) and κ reflects their differing sensitivities: minority-class accuracy evaluates performance exclusively on underrepresented classes, while κ

Table B.15
Parent-level classification results for the **EuroCropsML S2** benchmark. We report the parent-class classification accuracy on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per few-shot scenario is marked in **blue bold**. Black bold values highlight the best result for supervised or SSL pre-training. Additionally, cases in which the **LV+PT→EE** was an improvement over the corresponding **LV→EE** result are highlighted in green color.

(a) Parent-level 3 classification accuracy for fine-tuning runs on **EuroCropsML S2** where a prediction is considered correct if the predicted class shares the same parent category (HCAT3 level 3) as the ground truth.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.45 ± 0.11	0.586 ± 0.069	0.732 ± 0.062	0.72 ± 0.12	0.775 ± 0.031	0.7832 ± 0.0079	0.815 ± 0.013
	transfer learning (LV)	0.454 ± 0.033	0.483 ± 0.066	0.546 ± 0.020	0.648 ± 0.064	0.701 ± 0.029	0.723 ± 0.017	0.765 ± 0.026
	ANIL (LV)	0.47 ± 0.16	0.552 ± 0.071	0.66 ± 0.12	0.737 ± 0.062	0.778 ± 0.039	0.790 ± 0.016	0.824 ± 0.013
	MAML (LV)	0.50 ± 0.10	**0.599 ± 0.055**	0.658 ± 0.025	0.766 ± 0.044	**0.808 ± 0.011**	**0.822 ± 0.018**	**0.8340 ± 0.0055**
	FOMAML (LV)	0.51 ± 0.12	0.554 ± 0.035	**0.736 ± 0.039**	**0.770 ± 0.023**	0.769 ± 0.030	0.774 ± 0.014	0.8116 ± 0.0090
	TIML (encoder) (LV)	**0.58 ± 0.12**	0.59 ± 0.11	0.664 ± 0.064	0.749 ± 0.060	0.757 ± 0.043	0.778 ± 0.016	0.796 ± 0.013
	TIML (no encoder) (LV)	0.50 ± 0.13	0.577 ± 0.090	0.630 ± 0.071	0.727 ± 0.056	0.744 ± 0.044	0.758 ± 0.029	0.8085 ± 0.0086
	transfer learning (LV + PT)	0.4431 ± 0.0086	0.421 ± 0.020	0.57 ± 0.01	0.579 ± 0.085	0.678 ± 0.019	0.701 ± 0.026	0.784 ± 0.024
	ANIL (LV + PT)	0.501 ± 0.064	0.505 ± 0.025	0.639 ± 0.076	0.752 ± 0.040	0.771 ± 0.029	0.789 ± 0.018	0.8157 ± 0.0096
	MAML (LV + PT)	0.446 ± 0.025	0.522 ± 0.021	0.681 ± 0.072	0.730 ± 0.082	0.778 ± 0.015	0.781 ± 0.014	0.820 ± 0.012
	FOMAML (LV + PT)	0.457 ± 0.016	0.513 ± 0.055	0.660 ± 0.091	0.722 ± 0.065	0.761 ± 0.031	0.773 ± 0.026	0.817 ± 0.010
	TIML (encoder) (LV + PT)	0.474 ± 0.038	0.458 ± 0.080	0.617 ± 0.091	0.717 ± 0.084	0.763 ± 0.029	0.744 ± 0.051	0.800 ± 0.010
	TIML (no encoder) (LV + PT)	0.496 ± 0.037	0.516 ± 0.047	0.610 ± 0.027	0.707 ± 0.061	0.742 ± 0.046	0.741 ± 0.028	0.800 ± 0.014
	PRESTO	0.498 ± 0.025	**0.559 ± 0.088**	0.556 ± 0.083	0.652 ± 0.029	0.735 ± 0.046	**0.751 ± 0.039**	0.7930 ± 0.0071
	PRESTO-XTS	0.488 ± 0.024	0.484 ± 0.051	0.56 ± 0.10	0.542 ± 0.080	0.70 ± 0.13	0.720 ± 0.020	**0.8028 ± 0.0095**
	CrossPRESTO-XTS	**0.504 ± 0.055**	0.490 ± 0.020	**0.583 ± 0.060**	**0.68 ± 0.11**	**0.779 ± 0.020**	0.70 ± 0.13	0.793 ± 0.020

(b) Parent-level 4 classification accuracy for fine-tuning runs on **EuroCropsML S2** where a prediction is considered correct if the predicted class shares the same parent category (HCAT3 level 4) as the ground truth.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	no pre-training	0.243 ± 0.098	0.390 ± 0.068	0.532 ± 0.055	0.566 ± 0.059	0.622 ± 0.036	0.684 ± 0.012	0.726 ± 0.011
	transfer learning (LV)	0.231 ± 0.052	0.304 ± 0.053	0.355 ± 0.036	0.495 ± 0.081	0.585 ± 0.036	0.618 ± 0.030	0.669 ± 0.039
	ANIL (LV)	0.262 ± 0.082	0.373 ± 0.091	0.528 ± 0.060	0.615 ± 0.067	0.669 ± 0.045	0.691 ± 0.021	0.742 ± 0.013
	MAML (LV)	0.277 ± 0.077	0.344 ± 0.078	0.500 ± 0.033	**0.637 ± 0.051**	**0.681 ± 0.025**	**0.714 ± 0.040**	**0.745 ± 0.011**
	FOMAML (LV)	0.281 ± 0.072	0.354 ± 0.051	**0.543 ± 0.048**	0.590 ± 0.048	0.649 ± 0.029	0.669 ± 0.014	0.7141 ± 0.0086
	TIML (encoder) (LV)	**0.307 ± 0.092**	0.338 ± 0.092	0.422 ± 0.025	0.525 ± 0.044	0.605 ± 0.041	0.608 ± 0.044	0.696 ± 0.014
	TIML (no encoder) (LV)	0.29 ± 0.14	**0.39 ± 0.11**	0.466 ± 0.075	0.578 ± 0.057	0.641 ± 0.038	0.661 ± 0.022	0.7238 ± 0.0079
	transfer learning (LV + PT)	0.173 ± 0.014	0.2264 ± 0.0093	0.377 ± 0.091	0.411 ± 0.085	0.5365 ± 0.0061	0.589 ± 0.028	0.681 ± 0.024
	ANIL (LV + PT)	0.270 ± 0.051	0.326 ± 0.026	0.432 ± 0.093	0.610 ± 0.033	0.641 ± 0.037	0.690 ± 0.015	0.724 ± 0.011
	MAML (LV + PT)	0.235 ± 0.020	0.340 ± 0.022	0.478 ± 0.091	0.588 ± 0.081	0.654 ± 0.019	0.679 ± 0.018	0.726 ± 0.017
	FOMAML (LV + PT)	0.214 ± 0.026	0.322 ± 0.049	0.440 ± 0.064	0.557 ± 0.058	0.637 ± 0.026	0.673 ± 0.021	0.7240 ± 0.0076
	TIML (encoder) (LV + PT)	0.201 ± 0.038	0.2540 ± 0.0086	0.429 ± 0.064	0.524 ± 0.025	0.578 ± 0.014	0.592 ± 0.041	0.688 ± 0.033
	TIML (no encoder) (LV + PT)	0.272 ± 0.025	0.312 ± 0.029	0.439 ± 0.022	0.544 ± 0.051	0.638 ± 0.041	0.643 ± 0.020	0.716 ± 0.016
	PRESTO	0.199 ± 0.068	0.301 ± 0.095	0.321 ± 0.049	0.398 ± 0.055	0.569 ± 0.029	0.605 ± 0.057	0.6529 ± 0.0091
	PRESTO-XTS	0.129 ± 0.062	0.26 ± 0.11	0.319 ± 0.084	0.463 ± 0.032	0.585 ± 0.066	**0.607 ± 0.027**	**0.713 ± 0.019**
	CrossPRESTO-XTS	**0.215 ± 0.078**	**0.37 ± 0.13**	**0.402 ± 0.044**	**0.547 ± 0.047**	**0.595 ± 0.041**	0.565 ± 0.058	0.698 ± 0.018

Table B.16
Parent-level classification accuracy for the **EuroCropsML S1+S2+ERA5** benchmark. We report the parent-level classification accuracy on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per few-shot scenario is marked in **bold**.

(a) Parent-level 3 classification accuracy for fine-tuning runs on **EuroCropsML S1+S2+ERA5** where a prediction is considered correct if the predicted class shares the same parent category (HCAT3 level 3) as the ground truth.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	No pretraining	0.485 ± 0.024	0.519 ± 0.059	**0.531 ± 0.076**	0.530 ± 0.087	0.68 ± 0.11	0.704 ± 0.099	0.791 ± 0.019
	PRESTO	0.473 ± 0.019	**0.562 ± 0.053**	0.52 ± 0.11	**0.673 ± 0.050**	**0.725 ± 0.045**	**0.741 ± 0.031**	**0.807 ± 0.017**
	PRESTO-XTS	0.457 ± 0.050	0.447 ± 0.044	0.45 ± 0.13	0.58 ± 0.14	0.67 ± 0.11	0.69 ± 0.12	0.791 ± 0.017
	CrossPRESTO-XTS	**0.498 ± 0.011**	0.464 ± 0.040	0.493 ± 0.027	0.617 ± 0.085	0.66 ± 0.12	0.686 ± 0.028	0.795 ± 0.012

(b) Parent-level 4 red classification accuracy for fine-tuning runs on **EuroCropsML S1+S2+ERA5** where a prediction is considered correct if the predicted class shares the same parent category (HCAT3 level 4) as the ground truth.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
accuracy	No pretraining	**0.24 ± 0.17**	**0.449 ± 0.048**	**0.30 ± 0.10**	0.465 ± 0.021	0.482 ± 0.040	0.523 ± 0.029	0.626 ± 0.032
	PRESTO	0.173 ± 0.045	0.309 ± 0.077	0.320 ± 0.039	0.448 ± 0.030	**0.593 ± 0.034**	**0.627 ± 0.039**	0.706 ± 0.018
	PRESTO-XTS	0.22 ± 0.17	0.404 ± 0.073	0.321 ± 0.042	**0.484 ± 0.019**	0.558 ± 0.050	0.586 ± 0.064	**0.707 ± 0.022**
	CrossPRESTO-XTS	0.20 ± 0.17	0.189 ± 0.045	0.218 ± 0.020	0.426 ± 0.039	0.538 ± 0.047	0.578 ± 0.049	0.701 ± 0.016

captures overall class-wise agreement, penalizing both change alignment and skewed distributions. Hence, a model might achieve strong performance on underrepresented classes — resulting in high minority-class accuracy — but perform poorly on the majority class. Since

Table B.17
Detailed n-way k-shot s-step analysis of the 20-shot **EuroCropsML S2** benchmark task. The metric classification accuracy was evaluated for six different n-way k-shot s-step meta-learning variants per algorithm. We report the mean ± standard deviation over five repeated experimental runs in all cases. The best result per algorithm is marked in **bold**.

	n-way	4	4	4	10	10	10
	k-shot	1	1	1	10	1	10
	s-step	1	4	10	1	1	1
	algorithm						
accuracy	ANIL (LV)	0.405 ± 0.061	**0.473 ± 0.068**	0.439 ± 0.065	0.369 ± 0.083	0.403 ± 0.094	0.359 ± 0.072
	MAML (LV)	0.464 ± 0.060	**0.465 ± 0.067**	0.419 ± 0.041	0.417 ± 0.084	0.361 ± 0.078	0.463 ± 0.058
	FOMAML (LV)	**0.449 ± 0.030**	0.448 ± 0.060	0.403 ± 0.052	0.399 ± 0.045	0.436 ± 0.048	0.36 ± 0.10
	TIML (encoder) (LV)	**0.414 ± 0.055**	0.367 ± 0.062	0.372 ± 0.074	0.327 ± 0.090	0.376 ± 0.085	0.296 ± 0.061
	TIML (no encoder) (LV)	0.385 ± 0.063	0.391 ± 0.073	**0.433 ± 0.081**	0.379 ± 0.077	0.371 ± 0.050	0.340 ± 0.064
	ANIL (LV + PT)	0.355 ± 0.060	0.396 ± 0.068	0.377 ± 0.075	**0.462 ± 0.069**	0.374 ± 0.084	0.331 ± 0.050
	MAML (LV + PT)	0.379 ± 0.073	**0.44 ± 0.11**	0.347 ± 0.076	0.362 ± 0.081	0.388 ± 0.063	0.357 ± 0.037
	FOMAML (LV + PT)	0.408 ± 0.051	**0.410 ± 0.074**	0.332 ± 0.092	0.392 ± 0.069	0.378 ± 0.083	0.342 ± 0.051
	TIML (encoder) (LV + PT)	0.342 ± 0.093	0.319 ± 0.040	0.352 ± 0.078	0.375 ± 0.047	**0.394 ± 0.062**	0.366 ± 0.060
	TIML (no encoder) (LV + PT)	0.361 ± 0.030	**0.424 ± 0.048**	0.418 ± 0.094	0.314 ± 0.052	0.314 ± 0.059	0.356 ± 0.053

Table B.18
Detailed earning rate analysis of the 20-shot **EuroCropsML S2** benchmark task. The metric classification accuracy was evaluated for three different learning rate tuning variants per algorithm. We report the mean ± standard deviation on the test set over five repeated experimental runs in all cases, *cf.* Section 5.2. The best result per algorithm is marked in **bold**.

	lr tuning algorithm	only head	different head & backbone	same head & backbone
accuracy	no pre-training†	0.32 ± 0.18	**0.486 ± 0.017**	0.463 ± 0.035
	transfer learning (LV)	0.261 ± 0.082	0.325 ± 0.087	**0.353 ± 0.085**
	ANIL (LV)	0.373 ± 0.051	**0.478 ± 0.064**	0.445 ± 0.056
	MAML (LV)	0.386 ± 0.056	**0.493 ± 0.058**	0.463 ± 0.072
	FOMAML (LV)	0.420 ± 0.041	**0.466 ± 0.022**	0.451 ± 0.066
	TIML (encoder) (LV)	0.345 ± 0.083	0.402 ± 0.074	**0.421 ± 0.064**
	TIML (no encoder) (LV)	0.378 ± 0.080	**0.448 ± 0.062**	0.423 ± 0.067
	transfer learning (LV + PT)	0.171 ± 0.021	0.256 ± 0.071	**0.261 ± 0.095**
	ANIL (LV + PT)	0.365 ± 0.060	**0.470 ± 0.056**	0.406 ± 0.084
	MAML on LV + PT	0.387 ± 0.073	0.41 ± 0.10	**0.444 ± 0.097**
	FOMAML (LV + PT)	0.377 ± 0.096	**0.440 ± 0.058**	0.420 ± 0.058
	TIML (encoder) (LV + PT)	0.343 ± 0.024	0.397 ± 0.039	**0.418 ± 0.049**
	TIML (no encoder) (LV + PT)	0.352 ± 0.041	**0.439 ± 0.059**	0.439 ± 0.067
	PRESTO	0.279 ± 0.047	**0.294 ± 0.073**	0.286 ± 0.052
	PRESTO-XTS	**0.433 ± 0.023**	0.422 ± 0.062	0.338 ± 0.031
	CrossPRESTO-XTS	0.434 ± 0.041	**0.4735 ± 0.0096**	0.37 ± 0.11

Table B.19
Detailed learning rate analysis of the 20-shot **EuroCropsML S1+S2+ERA5** benchmark task. The classification accuracy was evaluated for three different learning rate tuning variants per algorithm. We report the mean ± standard deviation on the test set over five repeated experimental runs in all cases, *cf.* Section 5.2. The best result per algorithm is marked in **bold**.

	lr tuning algorithm	only head	different head & backbone	same head & backbone
accuracy	no pre-training	0.31 ± 0.14	0.377 ± 0.077	**0.428 ± 0.055**
	PRESTO	0.295 ± 0.059	**0.357 ± 0.063**	0.288 ± 0.041
	PRESTO-XTS	0.33 ± 0.13	**0.431 ± 0.062**	0.372 ± 0.081
	CrossPRESTO-XTS	0.178 ± 0.088	**0.346 ± 0.096**	0.297 ± 0.087

Cohen's kappa evaluates agreement across the full label distribution, the model's weak performance on the majority class would lead to substantially lower kappa scores, despite its strengths on rare classes.

As in Section 6, we report the best-performing configuration for each algorithm based on the metric on the validation set in Tables B.22 and B.23, presenting the mean and standard deviation over five repeated experimental runs.

B.5.1. *EuroCropsML S2*

As evaluated in Section 7.1, the MAML algorithms show superior behavior, showing the highest Cohen's kappa scores, with ANIL achieving the best overall performance for 500 shots with $\kappa = 0.556 \pm 0.01$. In contrast, despite demonstrating relatively strong accuracy in low ($k \leq 20$) few-shot tasks, the randomly initialized baseline model achieves a rather low Cohen's kappa score. While TIML (without encoder) shows competitive results for **LV→EE**, in particular TIML (with encoder) seems to struggle massively with the high-class imbalance when being trained on **LV+PT→EE**, indicating that the model is unable to discern the differences between various crop classes and instead demonstrating a proclivity for the majority class. This further confirms the results from Tables 4b and 5b, which suggested that both the baseline as well as the TIML (with encoder) may have mainly learned to predict the dominant crop class `pasture meadow grassland grass`, instead of effectively learning the distinctions between the various crop classes. Although pre-training on additional Portugal data usually decreases overall accuracy (*cf.* Table 4a), it often leads to increased κ scores and minority-class performance (*cf.* Table 4b) for meta-learning algorithms (*cf.* Table B.22). In particular, FOMAML shows improved Cohen's kappa scores (highlighted in green).

Nevertheless, most supervised methods outperform the SSL models in all scenarios (Table B.22), although the performance gap narrows to approximately 7% between the best meta-learning algorithms and PRESTO-XTS at 500 samples. PRESTO achieves higher scores than both PRESTO-XTS and CrossPRESTO-XTS in the 1- and 5-shot settings. Notably, among the SSL methods, CrossPRESTO-XTS attains the highest scores with 10, 20, and 100 samples with $\kappa_{10} = 0.122 \pm 0.067$, $\kappa_{20} =$

Table B.20
Additional results for the **EuroCropsML S2** benchmark. The postfix D represents models that only used deterministic masking strategies during pre-training, while L stands for a larger Transformer, as specified in Appendix B.4. We report the classification accuracy on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per task is marked in **bold**.

(a) SSL fine-tuning results for additional experiments on the full **EuroCropsML S2** task.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	PRESTO-XTS (D)	0.064 ± 0.060	0.22 ± 0.14	0.27 ± 0.14	0.403 ± 0.045	0.450 ± 0.042	0.453 ± 0.023	0.600 ± 0.024
	PRESTO-XTS (D, L)	**0.15 ± 0.18**	0.28 ± 0.18	0.31 ± 0.14	0.464 ± 0.014	0.470 ± 0.022	**0.500 ± 0.023**	**0.607 ± 0.014**
	CrossPRESTO-XTS (D)	0.065 ± 0.080	**0.38 ± 0.11**	0.32 ± 0.13	0.462 ± 0.017	0.450 ± 0.047	0.495 ± 0.029	0.5993 ± 0.0091
	CrossPRESTO-XTS (D, L)	0.15 ± 0.18	0.20 ± 0.17	**0.38 ± 0.13**	**0.4750 ± 0.0089**	**0.484 ± 0.019**	0.495 ± 0.026	0.589 ± 0.026

(b) SSL fine-tuning results for additional experiments on the **EuroCropsML S2** task excluding the majority class `pasture meadow grassland grass`.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	PRESTO-XTS (D)	0.061 ± 0.049	0.023 ± 0.017	0.017 ± 0.023	**0.049 ± 0.059**	0.13 ± 0.13	0.422 ± 0.035	0.501 ± 0.023
	PRESTO-XTS (D, L)	**0.113 ± 0.063**	**0.040 ± 0.063**	**0.036 ± 0.036**	0.022 ± 0.035	0.20 ± 0.20	0.22 ± 0.22	**0.510 ± 0.012**
	CrossPRESTO-XTS (D)	0.046 ± 0.039	0.0085 ± 0.0076	0.016 ± 0.012	0.012 ± 0.010	0.110 ± 0.089	**0.437 ± 0.030**	0.5076 ± 0.0076
	CrossPRESTO-XTS (D, L)	0.105 ± 0.060	0.034 ± 0.060	0.018 ± 0.021	0.031 ± 0.066	**0.22 ± 0.18**	0.21 ± 0.17	0.493 ± 0.020

Table B.21
Overall accuracy for additional experiments on the **EuroCropsML S1+S2+ERA5** benchmark. The postfix D represents models that only used deterministic masking strategies during pre-training, while L stands for a larger Transformer, as specified in Appendix B.4. We report the classification accuracy on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per task is marked in **bold**.

(a) SSL fine-tuning results for additional experiments on the full **EuroCropsML S1+S2+ERA5** task.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	PRESTO-XTS (D)	**0.331 ± 0.085**	0.24 ± 0.17	0.15 ± 0.11	0.302 ± 0.083	0.416 ± 0.036	0.39 ± 0.12	0.54 ± 0.11
	PRESTO-XTS (D, L)	0.25 ± 0.17	0.30 ± 0.18	0.20 ± 0.12	0.387 ± 0.080	0.426 ± 0.040	0.442 ± 0.096	0.563 ± 0.064
	CrossPRESTO-XTS (D)	0.266 ± 0.046	0.33 ± 0.13	0.279 ± 0.086	**0.444 ± 0.025**	0.451 ± 0.038	0.458 ± 0.032	**0.594 ± 0.039**
	CrossPRESTO-XTS (D, L)	0.28 ± 0.18	**0.33 ± 0.10**	**0.30 ± 0.17**	0.438 ± 0.056	**0.454 ± 0.039**	**0.475 ± 0.026**	0.592 ± 0.025

(b) SSL fine-tuning results for additional experiments on the **EuroCropsML S1+S2+ERA5** task excluding the majority class `pasture meadow grassland grass`.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
accuracy	PRESTO-XTS (D)	0.0019 ± 0.0037	0.013 ± 0.026	**0.072 ± 0.037**	0.034 ± 0.031	0.114 ± 0.092	0.29 ± 0.16	0.42 ± 0.18
	PRESTO-XTS (D, L)	**0.036 ± 0.061**	**0.030 ± 0.063**	0.057 ± 0.058	**0.038 ± 0.058**	**0.19 ± 0.13**	0.25 ± 0.16	0.39 ± 0.26
	CrossPRESTO-XTS (D)	0.0028 ± 0.0031	0.0027 ± 0.0017	0.020 ± 0.015	0.0043 ± 0.0044	0.013 ± 0.001	**0.385 ± 0.062**	**0.463 ± 0.096**
	CrossPRESTO-XTS (D, L)	0.027 ± 0.060	0.010 ± 0.014	0.009 ± 0.014	0.030 ± 0.053	0.059 ± 0.079	0.31 ± 0.18	0.459 ± 0.035

Table B.22
Results for Cohen's kappa on the **EuroCropsML S2** benchmark task. We report Cohen's kappa on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per few-shot scenario is marked in **blue bold**. Black bold values highlight the best result for supervised or SSL pre-training. Additionally, cases in which the **LV+PT→EE** was an improvement over the corresponding **LV→EE** result are highlighted in green color. The postfix D represents models that only used deterministic masking strategies during pre-training, while L stands for a larger Transformer, as specified in Appendix B.4.

	benchmark task (*k*-shot) algorithm	1	5	10	20	100	200	500
Cohen's κ	no pre-training	0.004 ± 0.033	0.094 ± 0.035	0.206 ± 0.047	0.25 ± 0.13	0.367 ± 0.058	0.466 ± 0.018	0.5292 ± 0.0041
	transfer learning (LV)	0.079 ± 0.043	0.124 ± 0.059	0.165 ± 0.079	0.266 ± 0.072	0.361 ± 0.053	0.399 ± 0.038	0.482 ± 0.053
	ANIL (LV)	0.073 ± 0.079	0.156 ± 0.020	0.22 ± 0.13	0.358 ± 0.040	**0.421 ± 0.040**	0.476 ± 0.028	**0.556 ± 0.010**
	MAML (LV)	0.106 ± 0.032	0.119 ± 0.029	**0.272 ± 0.021**	**0.362 ± 0.050**	0.406 ± 0.030	**0.479 ± 0.056**	0.534 ± 0.034
	FOMAML (LV)	0.069 ± 0.068	0.120 ± 0.028	0.224 ± 0.042	0.298 ± 0.051	0.383 ± 0.039	0.449 ± 0.016	0.5102 ± 0.0073
	TIML (encoder) (LV)	**0.113 ± 0.072**	0.139 ± 0.043	0.189 ± 0.039	0.266 ± 0.036	0.358 ± 0.029	0.370 ± 0.033	0.476 ± 0.019
	TIML (no encoder) (LV)	0.107 ± 0.052	**0.180 ± 0.043**	0.219 ± 0.024	0.314 ± 0.046	0.406 ± 0.026	0.425 ± 0.018	0.5393 ± 0.0064
	transfer learning (LV + PT)	0.0657 ± 0.0079	0.1093 ± 0.0074	0.173 ± 0.055	0.208 ± 0.061	0.298 ± 0.037	0.379 ± 0.030	0.481 ± 0.027
	ANIL (LV + PT)	0.1090 ± 0.0080	0.1547 ± 0.0092	0.215 ± 0.069	0.330 ± 0.034	0.396 ± 0.027	0.461 ± 0.019	0.519 ± 0.025
	MAML (LV + PT)	0.084 ± 0.045	0.140 ± 0.012	0.231 ± 0.064	0.320 ± 0.065	0.386 ± 0.028	0.458 ± 0.021	0.537 ± 0.015
	FOMAML (LV + PT)	0.082 ± 0.026	0.133 ± 0.023	0.197 ± 0.049	0.317 ± 0.042	0.381 ± 0.020	0.457 ± 0.025	0.522 ± 0.012
	TIML (encoder) (LV + PT)	0.068 ± 0.026	0.094 ± 0.021	0.148 ± 0.086	0.230 ± 0.088	0.332 ± 0.016	0.340 ± 0.016	0.459 ± 0.039
	TIML (no encoder) (LV + PT)	0.102 ± 0.023	0.137 ± 0.022	0.205 ± 0.017	0.316 ± 0.047	0.403 ± 0.047	0.432 ± 0.020	0.525 ± 0.011
	PRESTO	**0.055 ± 0.023**	**0.091 ± 0.028**	0.109 ± 0.029	0.166 ± 0.043	0.323 ± 0.026	0.366 ± 0.043	0.430 ± 0.010
	PRESTO-XTS	0.0046 ± 0.0037	0.053 ± 0.039	0.082 ± 0.055	0.056 ± 0.061	0.31 ± 0.17	0.379 ± 0.023	**0.518 ± 0.024**
	CrossPRESTO-XTS	0.020 ± 0.024	0.039 ± 0.041	**0.122 ± 0.067**	**0.2486 ± 0.0097**	**0.344 ± 0.038**	0.26 ± 0.14	0.496 ± 0.019
	PRESTO-XTS (D)	0.0067 ± 0.0067	0.036 ± 0.033	0.037 ± 0.034	0.080 ± 0.081	0.23 ± 0.13	0.360 ± 0.024	0.498 ± 0.024
	PRESTO-XTS (D, L)	0.000 039 ± 0.000 088	0.024 ± 0.022	0.061 ± 0.064	0.08 ± 0.11	0.21 ± 0.19	0.23 ± 0.21	0.504 ± 0.014
	CrossPRESTO-XTS (D)	0.008 ± 0.015	0.037 ± 0.039	0.089 ± 0.047	0.065 ± 0.033	0.273 ± 0.062	**0.396 ± 0.027**	0.4981 ± 0.0083
	CrossPRESTO-XTS (D, L)	0.000 57 ± 0.000 90	0.036 ± 0.040	0.057 ± 0.059	0.07 ± 0.13	0.26 ± 0.16	0.28 ± 0.16	0.485 ± 0.020

Table B.23
Results for Cohen's kappa on the **EuroCropsML S1+S2+ERA5** benchmark task. We report Cohen's kappa on the test set for each algorithm and few-shot task. Metrics are shown as mean ± standard deviation over five runs, *cf.* Section 5.2. The best result per task is marked in **bold**. The postfix D represents models that only used deterministic masking strategies during pre-training, while L stands for a larger Transformer, as specified in Appendix B.4.

	benchmark task (*k*-shot) / algorithm	1	5	10	20	100	200	500
Cohen's κ	no pre-training	-0.005 ± 0.013	0.032 ± 0.050	0.073 ± 0.034	0.043 ± 0.066	0.22 ± 0.13	0.31 ± 0.15	0.487 ± 0.034
	PRESTO	$\mathbf{0.038 \pm 0.013}$	$\mathbf{0.087 \pm 0.027}$	$\mathbf{0.128 \pm 0.033}$	$\mathbf{0.199 \pm 0.027}$	$\mathbf{0.352 \pm 0.028}$	$\mathbf{0.399 \pm 0.035}$	0.495 ± 0.018
	PRESTO-XTS	-0.003 ± 0.020	0.008 ± 0.030	0.066 ± 0.047	0.08 ± 0.11	0.28 ± 0.16	0.31 ± 0.18	$\mathbf{0.518 \pm 0.023}$
	CrossPRESTO-XTS	0.0005 ± 0.0029	0.033 ± 0.025	0.091 ± 0.010	0.145 ± 0.081	0.24 ± 0.13	0.361 ± 0.062	0.491 ± 0.021
	PRESTO-XTS (D)	0.000 ± 0.025	0.005 ± 0.015	0.062 ± 0.035	0.078 ± 0.070	0.19 ± 0.12	0.27 ± 0.13	0.42 ± 0.14
	PRESTO-XTS (D, L)	0.006 ± 0.010	0.007 ± 0.011	0.052 ± 0.046	0.109 ± 0.089	0.256 ± 0.080	0.28 ± 0.10	0.37 ± 0.25
	CrossPRESTO-XTS (D)	-0.011 ± 0.033	0.046 ± 0.024	0.084 ± 0.027	0.045 ± 0.020	0.051 ± 0.025	0.352 ± 0.034	0.478 ± 0.059
	CrossPRESTO-XTS (D, L)	0.006 ± 0.028	0.036 ± 0.052	0.024 ± 0.042	0.050 ± 0.086	0.20 ± 0.13	0.330 ± 0.092	0.484 ± 0.023

Table C.24
Run times for the considered supervised algorithms during the pre-training and 500-shot fine-tuning phases per run of the model training. For the meta-learning algorithms, we used the 4-way 1-shot 4-step variant. For the pre-training and fine-tuning, we report the run time of the training phase, excluding the tuning. The fastest run time per training phase is marked in **bold**.

benchmark task training phase algorithm / runtime	LV→EE pre-training (in hours)	LV→EE fine-tuning (in minutes)	LV+PT→EE pre-training (in hours)	LV+PT→EE fine-tuning (in minutes)
no pre-training	n/a	$\mathbf{3.8 \pm 1.9}$	n/a	11.7 ± 2.9
transfer learning	**0.73**	4.0 ± 1.2	**2.9**	10 ± 13
ANIL	3.8	5.8 ± 2.3	3.9	7.4 ± 3.0
MAML	5.3	5.1 ± 2.8	5.3	$\mathbf{5.9 \pm 2.4}$
FOMAML	4.1	7.4 ± 2.4	4.2	8.3 ± 7.5
TIML (encoder)	4.3	10.0 ± 3.5	4.3	19.9 ± 9.9
TIML (no encoder)	5.4	4.5 ± 1.9	5.7	17 ± 12

0.2486 ± 0.0097, and $\kappa_{100} = 0.344 \pm 0.038$. PRESTO-XTS, while initially underperforming, exhibits steady improvement with more data and outperforms the other SSL methods in the 500-shot regime. Moreover, the results on the (D, L) variants confirm the assumption from Appendix B.4 that, while improving overall accuracy, the models mainly focus on the majority class `pasture meadow grassland grass` instead of learning diverse class patterns. Training the baseline model on all **EuroCropsML S2** data, achieves $\kappa = 0.705 \pm 0.002$ on the test set.

B.5.2. EuroCropsML S1+S2+ERA5

A similar trend for SSL is observed in Table B.23, where the smaller PRESTO consistently achieves the highest κ scores in the 1- to 200-shot range and is only exceeded by PRESTO-XTS in the 500-shot setting with an increase of 4%. CrossPRESTO-XTS shows stable improvement with increasing sample sizes and closely follows the performance of PRESTO-XTS on 500 samples. The baseline model trained on all data of the **EuroCropsML S1+S2+ERA5** task yields $\kappa = 0.673 \pm 0.016$ on the test set.

Appendix C. Run-time analysis

Besides performance measures such as prediction accuracy, computational costs, and training times required to achieve performance are also of interest when benchmarking algorithms for their knowledge transfer capabilities. A complete and fully comparable run-time evaluation of all experiments considered in our benchmark is not possible, as the experimental runs were partially conducted in parallel on multiple machines with varying hardware specifications. Hence, we cannot present a joint run-time analysis of supervised and self-supervised learning algorithms.

Instead, for supervised methods, we present a run-time analysis of one pre-training variant per algorithm. Moreover, since meta-learning promises faster task adaptation, we also evaluate one fine-tuning task per algorithm. Table C.24 shows the model training times per algorithm during supervised pre-training and 500-shot fine-tuning. For meta-learning algorithms, we report the times of the 4-way 1-shot 4-step variant. All of these were performed under comparable circumstances on the same machine with 16 cores of AMD® EPYC™ 9654 CPU, 64 GB RAM, and an NVIDIA® GeForce RTX™ 4090 GPU.

The randomly initialized benchmark model has the shortest overall training time, because it does not require any pre-training and has the lowest fine-tuning times. While meta-learning algorithms of the MAML-family fulfill the promise of quicker task adaptation and, thus, in general, good fine-tuning times, this comes at significantly increased computational costs during pre-training. Altogether, the meta-learning approaches require longer total training times, with TIML (without encoder) being the most compute-heavy of all compared algorithms. Interestingly, the run times of the meta-learning algorithms are less affected by incorporating additional pre-training data from Portugal, while transfer learning takes significantly longer in this case. After pre-training also on Portugal data, for the fine-tuning phase, the MAML-family performs best.

To assess the computational efficiency of SSL methods, we compare PRESTO-XTS with CrossPRESTO-XTS, which is designed to reduce training costs during pre-training. On an AMD® EPYC™ 7763 CPU (64 cores), two NVIDIA® RTX™ 6000 Ada GPUs (48 GB each), and 512 GB of system RAM, CrossPRESTO-XTS achieves 13% higher throughput on average, processing each batch with 0.31 s instead of 0.36 s. Moreover, it also converges faster, typically triggering early stopping sooner. These improvements lead to an overall reduction in the average pre-training time of approximately 52%.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD). Association for Computing Machinery, pp. 2623–2631. http://dx.doi.org/10.1145/3292500.3330701.

Antonijević, O., Jelić, S., Bajat, B., Kilibarda, M., 2023. Transfer learning approach based on satellite image time series for the crop classification problem. J. Big Data 10 (1), 54. http://dx.doi.org/10.1186/s40537-023-00735-2.

Baik, S., Choi, M., Choi, J., Kim, H., Lee, K.M., 2020. Meta-learning with adaptive hyperparameters. In: Advances in Neural Information Processing Systems. NIPS '20, 33, Curran Associates Inc., Vancouver, BC, Canada, pp. 1–11.

Bekmukhamedov, N., Karabkina, N., Kurbanova, R., Koshim, G.A., Yegizbayeva, A., Ilyas, S., 2024. Advancing rice yield forecasting and crop assessment in Kazakhstan's Kyzylorda region: A multisource satellite data approach. In: Slimani, K.,

Gerasymov, O., Kerkeb, M. (Eds.), International Conference on Smart Technologies and Applied Research. STAR'2023, In: E3S Web of Conferences, 477, EDP Sciences, p. 00001. http://dx.doi.org/10.1051/e3sconf/202447700001.

Boussioux, L., Zeng, C., Guenais, T., Bertsimas, D., 2022. Hurricane forecasting: A novel multimodal machine learning framework. Weather. Forecast. 37 (6), 817–831. http://dx.doi.org/10.1175/WAF-D-21-0091.1.

Brown, C.F., Brumby, S.P., Guzder-Williams, B., Birch, T., Hyde, S.B., Mazzariello, J., Czerwinski, W., Pasquarella, V.J., Haertel, R., Ilyushchenko, S., Schwehr, K., Weisse, M., Stolle, F., Hanson, C., Guinan, O., Moore, R., Tait, A.M., 2022. Dynamic World, near real-time global 10 m land use land cover mapping. Sci. Data 9 (1), 251. http://dx.doi.org/10.1038/s41597-022-01307-4.

Chamorro Martinez, J., Feitosa, R., Cue La Rosa, L., Sanches, I., Nigri Happ, P., 2021. Fully convolutional recurrent networks for multidate crop recognition from multitemporal image sequences. ISPRS J. Photogramm. Remote Sens. 171, http://dx.doi.org/10.1016/j.isprsjprs.2020.11.007.

Cong, Y., Khanna, S., Meng, C., Liu, P., Rozi, E., He, Y., Burke, M., Lobell, D., Ermon, S., 2022. SatMAE: Pre-training transformers for temporal and multi-spectral satellite imagery. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (Eds.), Advances in Neural Information Processing Systems. 35, Curran Associates, Inc., pp. 197–211.

Dhande, A., Malik, R., 2022. Design of a highly efficient crop damage detection ensemble learning model using deep convolutional networks. J. Ambient. Intell. Humaniz. Comput. 14, 1–11. http://dx.doi.org/10.1007/s12652-022-04352-4.

Dumoulin, V., Houlsby, N., Evci, U., Zhai, X., Goroshin, R., Gelly, S., Larochelle, H., Larochelle, H., 2021. A unified few-shot classification benchmark to compare transfer and meta learning approaches. In: Vanschoren, J., Yeung, S. (Eds.), Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 1, pp. 1–14.

Ebel, P., Xu, Y., Schmitt, M., Zhu, X.X., 2022. SEN12MS-CR-TS: A remote-sensing data set for multimodal multitemporal cloud removal. IEEE Trans. Geosci. Remote Sens. 60, 1–14. http://dx.doi.org/10.1109/tgrs.2022.3146246.

Finn, C., Abbeel, P., Levine, S., 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In: Proceedings of the 34th International Conference on Machine Learning. ICML, JMLR.org, pp. 1126–1135.

Fu, L., Lian, L., Wang, R., Shi, B., Wang, X., Yala, A., Darrell, T., Efros, A.A., Goldberg, K., 2025. Rethinking patch dependence for masked autoencoders. Trans. Mach. Learn. Res. (TMLR) arXiv:2401.14391.

Fuller, A., Millard, K., Green, J.R., 2023. CROMA: Remote sensing representations with contrastive radar-optical masked autoencoders. In: Advances in Neural Information Processing Systems. Curran Associates, Inc., pp. 5506–5538.

Gómez, C., White, J.C., Wulder, M.A., 2016. Optical remotely sensed time series data for land cover classification: A review. ISPRS J. Photogramm. Remote. Sens. (P&rs) 116, 55–72. http://dx.doi.org/10.1016/j.isprsjprs.2016.03.008.

Hao, P., Di, L., Zhang, C., Guo, L., 2020. Transfer learning for crop classification with Cropland Data Layer data (CDL) as training samples. Sci. Total Environ. 733, 138869. http://dx.doi.org/10.1016/j.scitotenv.2020.138869.

He, K., Chen, X., Xie, S., Li, Y., Dollar, P., Girshick, R., 2022. Masked autoencoders are scalable vision learners. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, IEEE, pp. 15979–15988. http://dx.doi.org/10.1109/cvpr52688.2022.01553.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, IEEE, pp. 770–778. http://dx.doi.org/10.1109/CVPR.2016.90.

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., Nicolas, J., Peubey, C., Radu, R., Schepers, D., Simmons, A., Soci, C., Abdalla, S., Abellan, X., Balsamo, G., Bechtold, P., Biavati, G., Bidlot, J., Bonavita, M., De Chiara, G., Dahlgren, P., Dee, D., Diamantakis, M., Dragani, R., Flemming, J., Forbes, R., Fuentes, M., Geer, A., Haimberger, L., Healy, S., Hogan, R.J., Hólm, E., Janisková, M., Keeley, S., Laloyaux, P., Lopez, P., Lupu, C., Radnoti, G., de Rosnay, P., Rozum, I., Vamborg, F., Villaume, S., Thépaut, J.-N., 2020. The ERA5 global reanalysis. Q. J. R. Meteorol. Soc. 146 (730), 1999–2049. http://dx.doi.org/10.1002/qj.3803.

Hospedales, T., Antoniou, A., Micaelli, P., Storkey, A., 2022. Meta-learning in neural networks: A survey. IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI) 44 (9), 5149–5169. http://dx.doi.org/10.1109/TPAMI.2021.3079209.

Huisman, M., van Rijn, J.N., Plaat, A., 2021. A survey of deep meta-learning. Artif. Intell. Rev. 54 (6), 4483–4541. http://dx.doi.org/10.1007/s10462-021-10004-4.

Inácio, M., Pereira, V., Pinto, M., 2008. The soil geochemical atlas of Portugal: Overview and applications. J. Geochem. Explor. 98 (1), 22–33. http://dx.doi.org/10.1016/j.gexplo.2007.10.004.

Karklins, A., 2002. A comparative study of the Latvian soil classification with WRB. URL https://api.semanticscholar.org/CorpusID:5980982.

Kerner, H.R., Tseng, G., Becker-Reshef, I., Nakalembe, C., Barker, B., Munshell, B., Paliyam, M., Hosseini, M., 2020. Rapid response crop maps in data sparse regions. In: ACM SIGKDD Conference on Data Mining and Knowledge Discovery Workshops. arXiv:2006.16866.

Kingma, D.P., Ba, J., 2017. Adam: A method for stochastic optimization. In: International Conference on Learning Representations. ICLR, arXiv:1412.6980.

Kmoch, A., Kanal, A., Astover, A., Kull, A., Virro, H., Helm, A., Pärtel, M., Ostonen, I., Uuemaa, E., 2021. EstSoil-EH: A high-resolution eco-hydrological modelling parameters dataset for Estonia. Earth Syst. Sci. Data 13 (1), 83–97. http://dx.doi.org/10.5194/essd-13-83-2021.

Kumar, S., Torres, C., Ulutan, O., Ayasse, A., Roberts, D., Manjunath, B., 2020. Deep remote sensing methods for methane detection in overhead hyperspectral imagery. In: 2020 IEEE Winter Conference on Applications of Computer Vision. WACV, pp. 1765–1774. http://dx.doi.org/10.1109/WACV45572.2020.9093600.

Kussul, N., Lavreniuk, M., Skakun, S., Shelestov, A., 2017. Deep learning classification of land cover and crop types using remote sensing data. IEEE Geosci. Remote. Sens. Lett. PP, 1–5. http://dx.doi.org/10.1109/LGRS.2017.2681128.

Lee, K., Maji, S., Ravichandran, A., Soatto, S., 2019. Meta-learning with differentiable convex optimization. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, IEEE Computer Society, pp. 10649–10657. http://dx.doi.org/10.1109/CVPR.2019.01091.

Li, Z., Zhou, F., Chen, F., Li, H., 2017. Meta-SGD: Learning to learn quickly for few-shot learning. arXiv:1707.09835.

Marković, M., Živaljević, B., Mimić, G., Woznicki, S., Marko, O., Lugonja, P., 2023. Using Sentinel-1 data for soybean harvest detection in Vojvodina province, Serbia. In: Neale, C.M., Maltese, A. (Eds.), Remote Sensing for Agriculture, Ecosystems, and Hydrology XXV. In: Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series, 12727, SPIE, p. 127271G. http://dx.doi.org/10.1117/12.2679417.

Marszalek, M.L., Le Saux, B., Mathieu, P.-P., Nowakowski, A., Springer, D., 2022. Self-supervised learning - A way to minimize time and effort for precision agriculture? Int. Arch. Photogramm. Remote. Sens. Spat. Inf. Sci. XLIII-B3-2022, 1327–1333. http://dx.doi.org/10.5194/isprs-archives-XLIII-B3-2022-1327-2022.

Mishra, N., Rohaninejad, M., Chen, X., Abbeel, P., 2018. A simple neural attentive meta-learner. In: International Conference on Learning Representations. pp. 1–17, URL https://openreview.net/pdf?id=B1DmUzWAW.

Nichol, A., Achiam, J., Schulman, J., 2018. On first-order meta-learning algorithms. Comput. Res. Repos. (CoRR) arXiv:1803.02999.

Odenweller, J.B., Johnson, K.I., 1984. Crop identification using Landsat temporal-spectral profiles. Remote Sens. Environ. 14 (1), 39–54. http://dx.doi.org/10.1016/0034-4257(84)90006-3.

Ogidi, F.C., Eramian, M.G., Stavness, I., 2023. Benchmarking self-supervised contrastive learning methods for image-based plant phenotyping. Plant Phenomics 5, 0037. http://dx.doi.org/10.34133/plantphenomics.0037.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems (NeurIPS). 32, Curran Associates, Inc..

Qi, Y., Bitelli, G., Mandanici, E., Trevisiol, F., Application of deep learning crop classification model based on multispectral and SAR satellite imagery. Int. Arch. Photogramm. Remote. Sens. Spat. Inf. Sci. XLVIII-1/W2-2023, 1515–1521. http://dx.doi.org/10.5194/isprs-archives-XLVIII-1-W2-2023-1515-2023.

Rabus, B., Eineder, M., Roth, A., Bamler, R., 2003. The shuttle radar topography mission — A new class of digital elevation models acquired by spaceborne radar. ISPRS J. Photogramm. Remote Sens. 57 (4), 241–262. http://dx.doi.org/10.1016/S0924-2716(02)00124-7.

Raghu, A., Raghu, M., Bengio, S., Vinyals, O., 2019. Rapid learning or feature reuse? Towards understanding the effectiveness of MAML. In: International Conference on Learning Representations. ICLR, arXiv:1909.09157.

Reed, B.C., Brown, J.F., VanderZee, D., Loveland, T.R., Merchant, J.W., Ohlen, D.O., 1994. Measuring phenological variability from satellite imagery. J. Veg. Sci. 5 (5), 703–714. http://dx.doi.org/10.2307/3235884.

Reuss, J., Gikalo, E., Körner, M., 2025a. Mind the gap: Bridging prior shift in realistic few-shot crop-type classification. ISPRS Ann. Photogramm. Remote. Sens. Spat. Inf. Sci. arXiv:2511.16218.

Reuss, J., Macdonald, J., 2024. EuroCropsML. http://dx.doi.org/10.5281/zenodo.15095445.

Reuss, J., Macdonald, J., Becker, S., Richter, L., Körner, M., 2025b. The EuroCropsML time series benchmark dataset for few-shot crop type classification in Europe. Nat. Sci. Data 12, http://dx.doi.org/10.1038/s41597-025-04952-7.

Rouse, J.W., Haas, R.H., Schell, J.A., Deering, D.W., 1974. Monitoring vegetation systems in The Great Plains with ERTS. In: Third Earth Resources Technology Satellite-1 Symposium. 1, NASA, URL https://ntrs.nasa.gov/citations/19740022614.

Rußwurm, M., Körner, M., 2020. Self-attention for raw optical satellite time series classification. ISPRS J. Photogramm. Remote. Sens. (P&rs) 169, 421–435. http://dx.doi.org/10.1016/j.isprsjprs.2020.06.006.

Rußwurm, M., Pelletier, C., Zollner, M., Lefèvre, S., Körner, M., 2020a. BreizhCrops: A time series dataset for crop type mapping. In: ISPRS – International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences. XLIII-B2-2020, pp. 1545–1551. http://dx.doi.org/10.5194/isprs-archives-XLIII-B2-2020-1545-2020.

Rußwurm, M., Wang, S., Körner, M., Lobell, D., 2020b. Meta-learning for few-shot land cover classification. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops. CVPRW, p. 9. http://dx.doi.org/10.1109/cvprw50498.2020.00108.

Rusu, A.A., Rao, D., Sygnowski, J., Vinyals, O., Pascanu, R., Osindero, S., Hadsell, R., 2018. Meta-learning with latent embedding optimization. In: International Conference on Learning Representations. ICLR, arXiv:1807.05960.

Saini, R., Ghosh, S.K., Crop classification on single date Sentinel-2 imagery using Random Forest and Support Vector Machine. Int. Arch. Photogramm. Remote. Sens. Spat. Inf. Sci. XLII-5, 683–688. http://dx.doi.org/10.5194/isprs-archives-XLII-5-683-2018.

Sainte Fare Garnot, V., Landrieu, L., Giordano, S., Chehata, N., 2020. Satellite image time series classification with pixel-set encoders and temporal self-attention. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, pp. 12322–12331. http://dx.doi.org/10.1109/CVPR42600.2020.01234.

Santaga, F.S., Benincasa, P., Toscano, P., Antognelli, S., Ranieri, E., Vizzari, M., 2021. Simplified and advanced Sentinel-2-based precision nitrogen management of wheat. Agronomy 11 (6), http://dx.doi.org/10.3390/agronomy11061156.

Santoro, A., Bartunov, S., Botvinick, M., Wierstra, D., Lillicrap, T., 2016. Meta-learning with memory-augmented neural networks. In: Balcan, M.F., Weinberger, K.Q. (Eds.), Proceedings of the 33rd International Conference on Machine Learning. ICML, In: Proceedings of Machine Learning Research (PMLR), 48, PMLR, pp. 1842–1850.

Schmitt, M., Ahmadi, S.A., Hänsch, R., 2021. There is no data like more data - Current status of machine learning datasets in remote sensing. In: IEEE International Geoscience and Remote Sensing Symposium. IGARSS, pp. 1206–1209. http://dx.doi.org/10.1109/IGARSS47720.2021.9555129.

Schneider, M., Broszeit, A., Körner, M., 2021. EuroCrops: A pan-European dataset for time series crop type classification. In: Soille, P., Loekken, S., Albani, S. (Eds.), Conference on Big Data from Space (BiDS). Publications Office of the European Union, http://dx.doi.org/10.2760/125905.

Schneider, M., Chan, A., Körner, M., 2023a. EuroCrops. http://dx.doi.org/10.5281/zenodo.10118572.

Schneider, M., Körner, M., 2021. [Re] satellite image time series classification with pixel-set encoders and temporal self-attention. ReScience C 7 (19), http://dx.doi.org/10.5281/zenodo.4835356.

Schneider, M., Schelte, T., Schmitz, F., Körner, M., 2023b. EuroCrops: The largest harmonized open crop dataset across the European Union. Sci. Data 10 (1), 612. http://dx.doi.org/10.1038/s41597-023-02517-0.

Senaras, C., Holden, P., Davis, T., Rana, A.S., Grady, M., Wania, A., de Jeu, R., 2023. Self-supervised learning for crop classification using Planet Fusion. Int. Arch. Photogramm. Remote. Sens. Spat. Inf. Sci. XLVIII-M-1-2023, 309–315. http://dx.doi.org/10.5194/isprs-archives-XLVIII-M-1-2023-309-2023.

Service, N.A.S., 2024. National Agricultural Statistics Service Cropland Data Layer. Tech. rep., United States Department of Agriculture (USDA), URL https://www.nass.usda.gov/Research_and_Science/Cropland/SARS1a.php.

Shankar, S., Halpern, Y., Breck, E., Atwood, J., Wilson, J., Sculley, D., 2017. No classification without representation: Assessing geodiversity issues in open data sets for the developing world. In: Advances in Neural Information Processing Systems (NeurIPS) Workshops. p. 5, arXiv:1711.08536.

Simón Sánchez, A.-M., González-Piqueras, J., de la Ossa, L., Calera, A., 2022. Convolutional neural networks for agricultural land use classification from sentinel-2 image time series. Remote. Sens. 14 (21), http://dx.doi.org/10.3390/rs14215373.

Singh Behl, H., Güneş Baydin, A., Torr, P.H., 2019. Alpha MAML: Adaptive model-agnostic meta-learning. In: International Conference on Machine Learning (ICML) Workshops. arXiv:1905.07435.

Snell, J., Swersky, K., Zemel, R., 2017. Prototypical networks for few-shot learning. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. Curran Associates Inc., pp. 4080–4090, arXiv:1703.05175.

Song, X., Hansen, M.C., Potapov, P.V., Adusei, B., Pickering, J., Adami, M., Lima, A., Zalles, V., Stehman, S., Di Bella, C.M., Conde, M.C., Copati, E.J., Fernandes, L.B., Hernández-Serna, A., Jantz, S.M., Pickens, A.H., Turubanova, S., Tyukavina, A., 2021. Massive soybean expansion in South America since 2000 and implications for conservation. Nat. Sustain. 4, 784–792. http://dx.doi.org/10.1038/s41893-021-00729-z.

Stiller, S., Grahmann, K., Ghazaryan, G., Ryo, M., 2024. Improving spatial transferability of deep learning models for small-field crop yield prediction. ISPRS Open J. Photogramm. Remote. Sens. 12, 100064. http://dx.doi.org/10.1016/j.ophoto.2024.100064.

Sung, F., Yang, Y., Zhang, L., Xiang, T., Torr, P.H., Hospedales, T.M., 2018. Learning to compare: Relation network for few-shot learning. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 1199–1208, arXiv:1711.06025.

Thrun, S., Pratt, L., 1998. Learning to learn: Introduction and overview. In: Learning To Learn. Springer US, pp. 3–17. http://dx.doi.org/10.1007/978-1-4615-5529-2_1.

Triantafillou, E., Zhu, T., Dumoulin, V., Lamblin, P., Evci, U., Xu, K., Goroshin, R., Gelada, C., Swersky, K.J., Manzagol, P.-A., Larochelle, H., 2020. Meta-dataset: A dataset of datasets for learning to learn from few examples. In: International Conference on Learning Representations. ICLR, p. 24, arXiv:1903.03096, URL https://openreview.net/forum?id=rkgAGAVKPr.

Tseng, G., Cartuyvels, R., Zvonkov, I., Purohit, M., Rolnick, D., Kerner, H., 2023. Lightweight, pre-trained transformers for remote sensing timeseries. In: NeurIPS Workshop on Tackling Climate Change with Machine Learning: Blending New and Existing Knowledge Systems. arXiv:2304.14065.

Tseng, G., Kerner, H., Rolnick, D., 2022. TIML: Task-informed meta-learning for agriculture. arXiv:2202.02124.

Tseng, G., Zvonkov, I., Nakalembe, C., Kerner, H.R., 2021. CropHarvest: A global dataset for crop-type classification. In: Vanschoren, J., Yeung, S. (Eds.), Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 1, p. 14.

Tuia, D., Schindler, K., Demir, B., Camps-Valls, G., Zhu, X.X., Kochupillai, M., Džeroski, S., van Rijn, J.N., Hoos, H.H., Frate, F.D., Datcu, M., Quiané-Ruiz, J.-A., Markl, Saux, B.L., Schneider, R., Camps-Valls, G.V., 2024. Artificial intelligence to advance Earth observation: a perspective. IEEE Geosci. Remote. Sens. Mag. 2–25. http://dx.doi.org/10.1109/mgrs.2024.3425961.

Turkoglu, M.O., D'Aronco, S., Perich, G., Liebisch, F., Streit, C., Wegner, J.D., 2021. Crop mapping from image time series: Deep learning with multi-scale label hierarchies. Remote Sens. Environ. 264, 112603. http://dx.doi.org/10.1016/j.rse.2021.112603.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems (NeurIPS). 30, Curran Associates, Inc., p. 15, arXiv:1706.03762.

Vinyals, O., Blundell, C., Lillicrap, T., Kavukcuoglu, K., Wierstra, D., 2016. Matching networks for one shot learning. In: Advances in Neural Information Processing Systems (NeurIPS). Curran Associates Inc., pp. 3637–3645, arXiv:1606.04080.

Wang, H., Chang, W., Yao, Y., Yao, Z., Zhao, Y., Li, S., Liu, Z., Zhang, X., 2022. CC-SSL: A self-supervised learning framework for crop classification with few labeled samples. IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 15, 8704–8718. http://dx.doi.org/10.1109/JSTARS.2022.3211994.

Wang, H., Chang, W., Yao, Y., Yao, Z., Zhao, Y., Li, S., Liu, Z., Zhang, X., 2023. Cropformer: A new generalized deep learning classification approach for multi-scenario crop classification. Front. Plant Sci. 14, 1130659. http://dx.doi.org/10.3389/fpls.2023.1130659.

Wang, S., Rußwurm, M., Körner, M., Lobell, D.B., 2020. Meta-learning for few-shot time series classification. In: IEEE International Geoscience and Remote Sensing Symposium. IGARSS, pp. 7041–7044. http://dx.doi.org/10.1109/IGARSS39084.2020.9441016.

Wang, A.X., Tran, C., Desai, N., Lobell, D., Ermon, S., 2018. Deep transfer learning for crop yield prediction with remote sensing data. In: ACM Conference on Computing and Sustainable Societies. Association for Computing Machinery, pp. 1–5. http://dx.doi.org/10.1145/3209811.3212707.

Ye, H.-J., Chao, W.-L., 2021. How to train your MAML to excel in few-shot classification. In: International Conference on Learning Representations (ICLR). arXiv:2106.16245.

Yoon, J., Kim, T., Dia, O., Kim, S., Bengio, Y., Ahn, S., 2018. Bayesian model-agnostic meta-learning. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (Eds.), Advances in Neural Information Processing Systems (NeurIPS). 31, Curran Associates, Inc., p. 11.

Yuan, Y., Lin, L., 2021. Self-supervised pretraining of transformers for satellite image time series classification. IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 14, 474–487. http://dx.doi.org/10.1109/JSTARS.2020.3036602.

Yuan, Y., Lin, L., Liu, Q., Hang, R., Zhou, Z.-G., 2022. SITS-Former: A pre-trained spatio-spectral-temporal representation model for Sentinel-2 time series classification. Int. J. Appl. Earth Obs. Geoinf. 106, 102651. http://dx.doi.org/10.1016/j.jag.2021.102651.